SensOFormer: Foundation Models for Particle Accelerators

Mahindra Rautela, Alexander Scheinker

Los Alamos National Laboratory, Los Alamos, NM, USA.
{mrautela,ascheink}@lanl.gov

Abstract

Large-scale scientific facilities, such as particle accelerators, are complex systems composed of thousands of interacting components that must operate collectively to support diagnostics and control, and ensure safe, reliable operation. These facilities rely on extensive sensor networks that generate large volumes of heterogeneous, incomplete, and noisy telemetry, often sampled at different rates across subsystems. In this paper, we adopt the foundation-model paradigm to learn general-purpose sensor representations from historical accelerator data through self-supervised pretraining. We introduce SensOFormer, a denoising masked transformer with a Perceiver-style encoder–decoder architecture designed to handle variable sensor sets, accommodate heterogeneous sampling rates, and learn robust representations from noisy measurements. The model is pretrained on multiple particle-accelerator datasets and transferred to downstream tasks, including missing-data imputation, anomaly detection, cavity identification, and fault identification. Through extensive evaluations and comparisons, we demonstrate that a single self-supervised model can learn reusable representations for diverse operational tasks in large-scale scientific facilities. **Project page**: `https://mahindrautela.github.io/sensoformer`.

1 Introduction

Particle accelerators are large-scale scientific facilities that enable fundamental studies in particle and nuclear physics, materials characterization using neutron and X-ray scattering, ultrafast imaging of chemical and biological processes, isotope production, radiation-effects testing, and high-energy-density science. Delivering these capabilities requires thousands of tightly coupled components to operate under strict stability, safety, and reliability constraints. Modern accelerators are therefore instrumented with extensive sensors and control systems that continuously monitor beam conditions, radio-frequency systems, magnets, vacuum, cooling, timing, and other subsystems [Chao et al., 2023]. *These instruments generate massive streams of operational telemetry that are noisy, incomplete, heterogeneous, and distributed across the facility*. As a result, core operational tasks such as missing-data recovery, anomaly detection, machine tuning, and control decision support depend critically on learning useful structure from high-dimensional sensor measurements [Edelen and Huang, 2024].

A foundation model to learn generalized sensor representations. The abundance of accelerator telemetry motivates a shift from training separate task-specific models for individual diagnostics, tuning, or control problems to learning general-purpose representations from large-scale unlabeled operational data. Foundation models are pretrained on broad, largely unlabeled data using self-supervised objectives and then adapted to many downstream tasks [Bommasani et al., 2021]. This recipe has driven progress in language modeling [Devlin et al., 2019, Radford et al., 2019], masked visual representation learning [He et al., 2022], time-series modeling [Das et al., 2024, Ansari et al., 2024], and physical spatiotemporal modeling [Bodnar et al., 2025, Rautela et al., 2025]. Particle accelerators present an analogous opportunity: years of archived process variables, control signals,

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

and beam diagnostics encode recurring machine states, subsystem couplings, operational drifts, and rare failure modes, yet most of these data are weakly labeled or unlabeled [Edelen and Huang, 2024]. *A self-supervised accelerator foundation model can therefore turn routine operational data into a learning signal, producing reusable representations for pressing accelerator problems such as imputation, anomaly detection, cavity identification and fault identification*. Task-specific models remain suitable for fixed sensor configurations with sufficient labeled data. However, a shared pretrained backbone enables lightweight adaptation across diagnostic tasks, reducing repeated model development and potentially improving label efficiency when annotated fault data are scarce.

Challenges with operational data. Accelerator telemetry differs substantially from curated multivariate time-series benchmarks [Li et al., 2023]. Operational data are inherently *heterogeneous*, collected from many subsystem-specific diagnostics, including superconducting RF cavity waveforms, RF station phase and amplitude signals, beam-position monitors, beam-loss monitors, magnet and power-supply variables, and high-voltage converter modulator waveforms [Tennant et al., 2020, St. John et al., 2021, Krymova et al., 2023, Liang et al., 2025, Radaideh et al., 2022]. Each data acquisition (DAQ) segment may contain hundreds to thousands of channels, and at facility scale the number of available process variables can be much larger [St. John et al., 2021]. Moreover, the available sensors, their *ordering*, and their *grouping* can vary across sectors, subsystems, and acquisition periods. Consequently, the *semantic identity* of a channel is determined by sensor metadata, physical location, and subsystem membership rather than by its column index [Krymova et al., 2023]. Models that assume a fixed channel layout can therefore be brittle under sensor reordering, missing channels, or newly added diagnostics. Padding each acquisition to a facility-wide maximum channel set is also computationally inefficient at this scale. Second, accelerator measurements are *asynchronous*: sensors are sampled at different rates, logged over variable-length windows, and interrupted by dropouts, maintenance periods, and DAQ-specific failures [Tennant et al., 2020]. Third, accelerators are *time-varying systems* whose operating distributions drift with beam mode, machine configuration, component aging, calibration changes, and seasonal or operational conditions [Scheinker et al., 2023]. Finally, the data are *noisy* with noise statistics that are difficult to specify a priori [Liang et al., 2025]. Taken together, these properties make fixed-dimensional, task-specific modeling pipelines difficult to scale and prone to poor generalization across sensors, subsystems, and operating conditions. Instead, accelerator telemetry calls for models that can ingest variable numbers of sensor observations without padding, treat channels as an unordered or metadata-indexed set, handle missing and irregularly sampled measurements, and learn denoising representations directly from large-scale operational data.

Model design requirements and SensOFormer. In light of the above-mentioned challenges, we design both the foundation model architecture and the pretraining strategy to accommodate heterogeneous accelerator telemetry. *SensOFormer is a latent transformer model with a set encoder and set decoder*. The set encoder, inspired by perceiver-style models [Jaegle et al., 2021], is *permutation-invariant* across channels, allowing the model to process variable sensor sets without relying on fixed column semantics [Lee et al., 2019]. The set decoder is *permutation-equivariant* across channels, preserving the batch-specific sensor locations and identities of the queried outputs. Together, the encoder, latent transformer, and decoder provide flexibility across both channels and sequence lengths without requiring data or activation padding. This design is important for scaling to large accelerator datasets, where padding to the maximum number of sensors and time steps would be computationally inefficient. Finally, transformer-based architectures provide a scalable representation-learning framework, and the masked pretraining objective encourages the model to learn robust structure from noisy observations rather than overfitting to sensor-specific noise [Park and Kim, 2022, Wang et al., 2022].

Contributions. We make the following contributions with this research:

1. We introduce SensOFormer, to the best of our knowledge the *first foundation model designed specifically for particle-accelerator operational telemetry*.
2. We pretrain SensOFormer on a diverse multi-facility benchmark spanning LANSCE, LHC, SNS, LCLS, and CEBAF datasets. These datasets cover operational telemetry, beam-loss monitoring, RF station signals, high-voltage converter modulator waveforms, and superconducting RF cavity fault data.
3. We characterize scaling behavior with respect to model capacity and pretraining data volume. The scaling study shows that validation reconstruction loss improves with both larger models

and more data, and provides guidance for compute-efficient scaling of future accelerator foundation models.

4. We demonstrate strong zero-shot missing-data imputation across heterogeneous accelerator datasets, including transfer to a different year dataset (LANSCE-2025) and unseen accelerator facility (GMPS-Booster-Fermilab). We further show that pretrained SensOFormer representations can be adapted with lightweight task-specific heads for anomaly detection, cavity identification, and fault identification.

2 Related Work

Scientific ML in particle accelerators. Machine learning has become an increasingly important tool for particle accelerators, with applications in beam-dynamics modeling, diagnostics, optimization, control, tuning, physics-constrained simulation, and uncertainty quantification [Rautela et al., 2024, Edelen and Huang, 2024, Scheinker and Williams, 2025, Scheinker, 2025, Saxena et al., 2025, Roussel et al., 2021, Edelen et al., 2020, Wan et al., 2025, Leon and Scheinker, 2024, Mishra et al., 2021, Garcia-Cardona and Scheinker, 2024, Rautela and Scheinker, 2025]. In operational settings, ML has enabled adaptive virtual diagnostics at Los Alamos Neutron Science Center (LANSCE at Los Alamos National Laboratory) [Scheinker, 2021, Scheinker and Williams, 2025], superconducting RF cavity-fault localization and classification at Continuous Electron Beam Accelerator Facility (CEBAF at Jefferson Laboratory) [Tennant et al., 2020], anomaly detection in high-voltage converter modulator signals at Spallation Neutron Source (SNS at Oak Ridge National Laboratory) [Radaideh et al., 2022], beam-loss prediction at the Large Hadron Collider (LHC at CERN) [Krymova et al., 2023], RF station fault identification at Linac Coherent Light Source (LCLS at Stanford Linear Accelerator Laboratory) using beam-position-monitor and high-frequency RF phase data [Humble et al., 2022, Liang et al., 2025], and accelerator control at Fermilab [St. John et al., 2021]. While these efforts show the effectiveness of specialist ML models, accelerator facilities share recurring RF, magnet, vacuum, timing, diagnostic, and control subsystems monitored by related sensor classes. SensOFormer leverages common operational and subsystem organization across facilities by learning transferable accelerator sensor representations through self-supervised foundation-model pretraining, rather than training task-specific models.

Scientific foundation models. SensOFormer adopts the foundation-model paradigm to learn reusable sensor representations from accelerator operational telemetry. This paradigm has been established in language modeling [Devlin et al., 2019, Radford et al., 2019] and computer vision [He et al., 2022, Caron et al., 2021], and has recently gained momentum across scientific domains, including physical systems [Rautela et al., 2025, Park et al., 2025, Nguyen et al., 2023], chemical science [Wadell et al., 2025], biological science [Moor et al., 2023, Brixi et al., 2026], and time-series prediction [Ansari et al., 2024, Nie et al., 2022]. Across these domains, foundation models succeed by scaling self-supervised pretraining over large, diverse, and heterogeneous datasets, while using input representations flexible enough to accommodate missing, irregular, or modality-specific structure. Accelerator telemetry exhibits these same properties: it is abundant but weakly labeled, heterogeneous across subsystems and facilities, and corrupted by noise and affected by changing operating conditions. SensOFormer brings the foundation-model paradigm to this setting through denoising masked reconstruction over multivariate sensor sets, enabling transferable representations for downstream tasks such as missing-data imputation, anomaly detection, cavity and fault identification. A detailed comparison of SensOFormer with existing set-based and any-variate time-series models, including SeFT [Horn et al., 2020] and MOIRAI [Woo et al., 2024] is provided in Appendix A.

3 Proposed Method

3.1 Pretraining strategy

Before describing the model architecture, we first motivate the pretraining objective. Foundation models are commonly pretrained using either autoregressive prediction, such as next-token, next-step, or next-frame prediction [Radford et al., 2019, Ansari et al., 2024, Rautela et al., 2025], or masked reconstruction and denoising objectives [Devlin et al., 2019, Nie et al., 2022, Wadell et al., 2025]. Autoregressive forecasting is a natural pretraining choice when the primary goal is to extrapolate a sequence forward in time, as in language modeling, time-series forecasting for

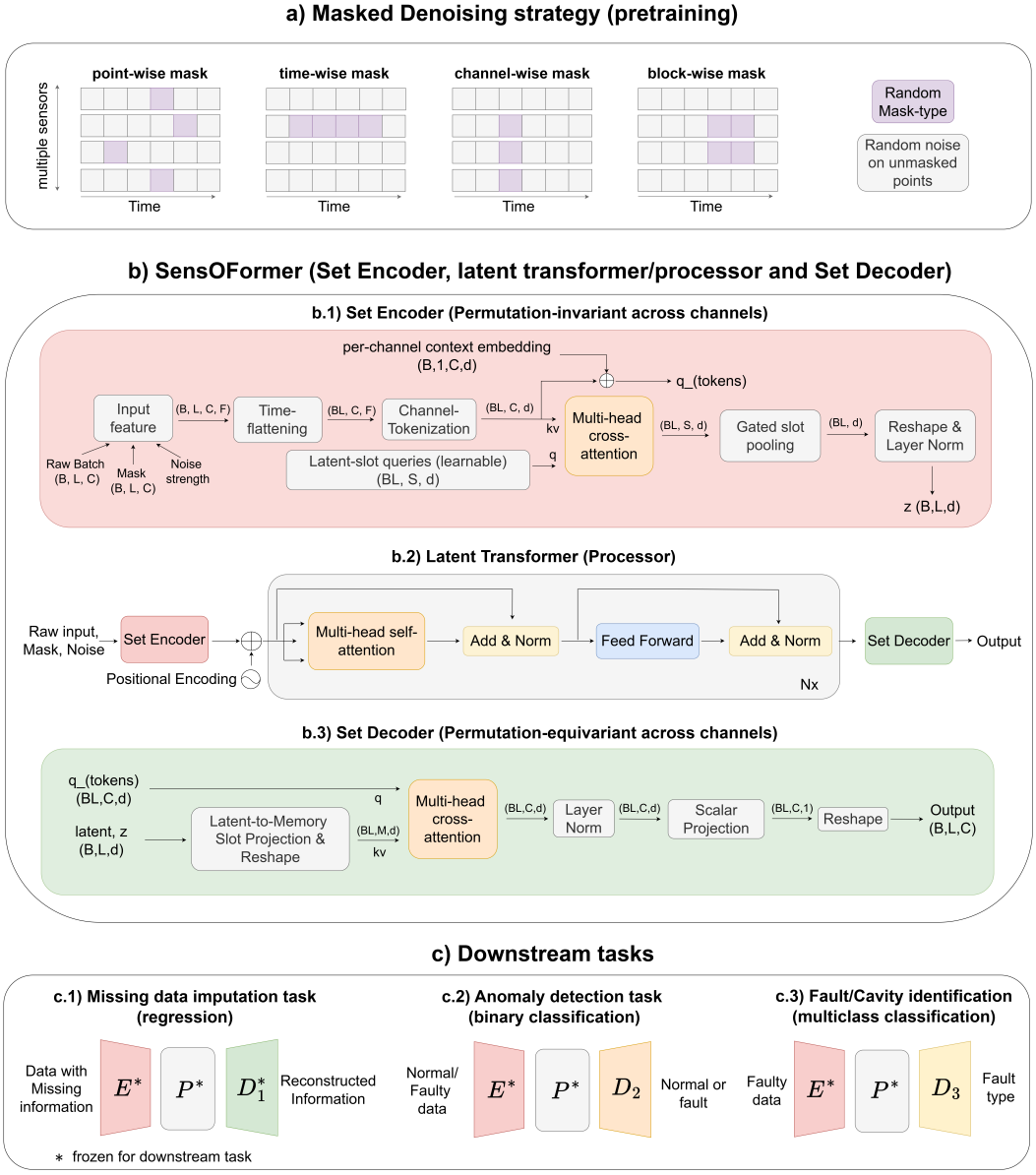


Figure 1: **Overview of SensOFormer.** (a) Self-supervised pretraining uses masked denoising reconstruction. A mask is sampled from a mask bank containing point-wise, time-wise, channel-wise, and block-wise masks, while noise is added to the unmasked values. (b) SensOFormer uses an encoder–processor–decoder architecture for variable sensor sets. The perceiver-style set encoder is permutation-invariant across sensor channels. The latent transformer processor models temporal dependencies. The set decoder is permutation-equivariant, producing channel-wise reconstructions that preserve the queried sensor identities. (c) The pretrained encoder–processor backbone is reused with task-specific heads for missing-data imputation, anomaly detection, and fault identification.

financial, energy, or traffic data, and physical systems with temporally evolving fields such as atmosphere and climate. Accelerator operational telemetry, however, does not generally form a single uniform sequence governed by one dominant time scale. Beam-related signals can correspond to fast machine events, since charged-particle bunches travel near the speed of light, whereas other subsystems, such as cooling, vacuum, and power-supply systems, evolve over slower operational time scales. Moreover, a central downstream task in this work is to recover missing or corrupted sensor measurements, rather than only to extrapolate future time steps. These considerations make masked denoising reconstruction a more natural pretraining objective for accelerator telemetry: the model must infer hidden measurements from the available temporal and cross-channel context while learning representations robust to noise, missingness, and heterogeneous sensor structure.

We therefore *pretrain SensOFormer using a denoising masked reconstruction objective*, illustrated in Figure 1(a). For each training instance, we sample a corruption pattern from a mask bank and hide the corresponding subset of sensor measurements. The mask bank includes point-wise, time-wise, channel-wise, and block-wise masks, which emulate common failure modes in accelerator telemetry such as isolated dropouts, missing time intervals, unavailable sensor channels, and structured channel–time gaps. The visible entries are additionally perturbed with additive noise sampled from multiple noise strengths, including zero, while the mask indicator specifies which entries have been hidden. Given a corrupted input batch and the corresponding binary mask, SensOFormer is trained to reconstruct the original sensor values at the masked locations. By combining structured masking with denoising, the pretraining task discourages simple copying of clean observations and instead encourages the model to learn robust temporal and cross-channel structure from noisy, incomplete accelerator data.

3.2 Overview of SensOFormer

Figure 1(b) illustrates SensOFormer as an encoder–processor–decoder architecture for accelerator sensor sets. The key design choice is to represent the sensors observed at each time step as an unordered set rather than as a fixed channel vector. This allows the model to handle variable sensor availability, channel ordering, and subsystem-specific measurements without tying the representation to a particular global sensor layout. Given a corrupted input batch and its mask indicator, SensOFormer forms per-channel features $x \in \mathbb{R}^{B\times L\times C\times F}$, where B is the batch size, L is the sequence length, C is the number of sensor channels, and F is the feature dimension. In our implementation, $F = 2$, corresponding to the corrupted sensor value and a binary mask indicator. The batch and time dimensions are flattened to obtain (BL, C, F), so that each time step is treated as a set of C channel observations. A shared tokenization layer maps these features to channel tokens in $\mathbb{R}^{BL\times C\times d}$.

The *set encoder* uses a Perceiver-style cross-attention mechanism in which S learnable latent queries attend to the channel tokens. Because the same attention operation is applied to an unordered set of sensor tokens, the encoder is permutation-invariant with respect to channel ordering: reordering the input sensors does not change the aggregated latent representation. The resulting slot representations are pooled and reshaped into a latent sequence $z \in \mathbb{R}^{B\times L\times d}$, which is processed by a transformer to model temporal dependencies in a compact shared latent space.

The *set decoder* maps the processed latent sequence back to channel-wise predictions. For each latent vector $z_{b,t}$, the decoder constructs M memory slots, and the channel tokens are reused as output queries that cross-attend to these memory slots. This produces one prediction per queried channel, followed by layer normalization and scalar projection to obtain $\hat{x} \in \mathbb{R}^{B\times L\times C}$. The decoder is permutation-equivariant: reordering the output channel queries reorders the corresponding predictions in the same way. Thus, *SensOFormer combines permutation-invariant encoding, temporal latent processing, and permutation-equivariant decoding to model variable sensor sets without requiring a fixed channel layout*.

3.3 Downstream tasks

Figure 1(c) summarizes how SensOFormer is adapted to downstream tasks after pretraining. We consider three task instantiations: missing-data imputation, anomaly detection, and fault identification. In all cases, the set encoder E and transformer processor P provide a shared representation of the operational sensor data, while the task-specific decoder or head D_i determines the output type. This

design allows the same pretrained backbone to support reconstruction, binary classification, and multiclass classification tasks.

For *missing-data imputation*, the input is partially observed telemetry in which missingness can occur as missing sensor values, consecutive missing time steps, missing channel values over multiple time steps, or structured channel–time blocks. Simple interpolation or local averaging can be effective for isolated missing points, but becomes unreliable when multiple time windows or sensor channels are absent. As shown in Fig. 1(c.1), SensOFormer uses the reconstruction decoder D_1 to predict the missing values from the available temporal and cross-channel context. This task is directly aligned with the denoising masked reconstruction objective used during pretraining, so the architecture can be reused for missing data reconstruction.

For *anomaly detection*, the goal is to distinguish normal operation from abnormal or faulty machine behavior using operational sensor data. This task is challenging because labeled anomalies are limited: fault annotation requires domain expertise and careful post-hoc analysis. Reliable anomaly detection is operationally important because accelerator facilities run under scheduled beam-time periods and can be interrupted by planned or unplanned maintenance. As shown in Fig. 1(c.2), we adapt the pretrained encoder–processor backbone with a binary classification head D_2 to identify normal versus faulty operating patterns.

For *fault identification*, the objective is to classify the type or source of a fault once faulty data are available. This is a more detailed multiclass classification problem than binary anomaly detection, requiring the model to distinguish between different fault categories or faulty components. As shown in Fig. 1(c.3), we reuse the same pretrained encoder E and processor P, and attach a task-specific multiclass head D_3 to predict the fault label.

4 Experiments

4.1 Datasets

The datasets used in this study span diverse accelerator facilities, sensor modalities, and operational tasks. They include multi-subsystem operational measurements from LANSCE at LANL [Scheinker, 2025], beam-loss measurements from LHC at CERN [Krymova et al., 2023], high-voltage converter modulator (HVCM) waveforms for power-electronics fault prognosis at SNS at ORNL [Radaideh et al., 2023b,a], superconducting radio-frequency (SRF) cavity fault data from CEBAF at JLab [Tennant et al., 2020], RF-station anomaly data from LCLS at SLAC [Humble et al., 2022, Liang et al., 2025], and GMPS control data from Booster at Fermilab [St. John et al., 2021]. This collection provides a heterogeneous benchmark for testing whether a single self-supervised sensor model can generalize across facilities, subsystems, sampling structures, and downstream tasks.

Table 1 summarizes the tensorized form of each dataset and the downstream tasks used in our evaluation. The datasets differ substantially in both temporal length and channel structure: LANSCE contains variable-channel operational telemetry, LHC-BL provides very long beam-loss time series, GMPS-Booster-Fermilab provides long-duration, multivariate operational telemetry capturing cycle-by-cycle readings and control settings, RFS-LCLS and HVCM-SNS contain fixed-channel waveform-style records, and SRF-CEBAF consists of short RF fault-event windows. All datasets are used for missing-data imputation (MDI), while HVCM-SNS and SRF-CEBAF additionally support diagnostic tasks such as anomaly detection, cavity identification, and fault identification. This design allows us to evaluate both zero-shot reconstruction across heterogeneous sensor formats and transfer to task-specific accelerator diagnostics.

4.2 Main results

For readability and conciseness, we defer details of the experimental settings, including model variants, windowing, masking, normalization, noise augmentation, the training objective, optimization, and compute, to Appendix B.

A detailed *scaling study* in Appendix C shows that validation masked-reconstruction loss decreases with both model size and pretraining data volume, with the best fitted point occurring at the largest evaluated model–data scale. In our setting, the fitted exponents are $\alpha = 0.715$ for model size and $\beta = 0.374$ for data size, indicating that both larger models and larger pretraining corpora

Table 1: Datasets used for pretraining and downstream evaluation. The benchmark spans operational telemetry and subsystem-specific diagnostics from multiple accelerator facilities. Tensor shapes are reported as (N, T, C), where N is the number of examples, T is the number of time steps, and C is the number of sensors/channels. MDI denotes missing-data imputation, and N^* denotes a variable number of samples and C^* denotes a variable number of channels, with the bracketed range indicating the minimum and maximum number of channels across the dataset. LANSCE-2025 and GMPS-Booster-Fermilab are evaluation-only, not included in pretraining.

Dataset	Facility / subsystem	Dataset/Tensor shape (N,T,C)	Downstream task
LANSCE (2019–2024)	LANSCE operational data	(4250, 10000, $C^* \in [1, 1754]$)	MDI on 10% test set
LHC-BL (2017) LHC-BL (2018)	LHC beam-loss monitoring	(1, 542061, 66) (1, 524430, 66)	MDI on 10% test set
RFS-LCLS (2020)	LCLS RF station signals (AMM and beam) LCLS RF station signals (AMPL and beam)	(1269, 2196, 9) (3208, 2076, 9)	MDI on 10% test set
RFS-LCLS (2024)	LCLS RF station signals (phase and beam)	(5364, 1066, 9)	MDI on 10% test set
HVCM-SNS	SNS high-voltage converter modulators	(10000, 3753, 12) (N^*, 3753, 12)	1. MDI on 10% test set 2. Anomaly detection
SRF-CEBAF	CEBAF superconducting RF cavities	(2375, 192, 1)	1. MDI on 10% test set 2. Cavity identification 3. Fault identification
LANSCE (2025)	LANSCE operational data	(17, 10000, $C^* \in [1, 769]$)	MDI on 100% test set
GMPS-Booster-Fermilab	Gradient Magnet Power Supply	(1,1295713, 43)	MDI on 100% test set

improve reconstruction loss within the evaluated regime. The resulting compute-optimal trend allocates additional compute to both dimensions, with data size increasing faster than parameter count, suggesting that future accelerator foundation models should scale the pretraining corpus alongside model capacity.

The main text focuses on three empirical questions that directly test whether SensOFormer learns reusable accelerator representations: (i) can the pretrained model perform zero-shot missing-data imputation across heterogeneous datasets spanning six accelerator facilities? (ii) can the frozen pretrained backbone support early anomaly detection on HVCM-SNS with only a lightweight supervised head? and (iii) can the same representation transfer to cavity and fault identification on SRF-CEBAF? We report these results in the following subsections.

4.2.1 Zero-shot missing-data imputation

Table 2 reports zero-shot point-wise imputation performance across datasets, mask ratios, and model variants: XTi (2.38M), Ti (5.80M), Mi (13.12M), S (35.87M), and B (80.54M). Lower values indicate better reconstruction. Overall, the B model achieves the lowest error in 25 of the 28 dataset–mask combinations, while the S model is best in the remaining three settings. This shows a clear benefit from increasing model capacity for zero-shot imputation, suggesting that larger SensOFormer variants more effectively use temporal and cross-channel context to reconstruct missing sensor values.

The *LANSCE-2025* setting is more challenging than the LANSCE 2019–2024 held-out test set because it evaluates transfer to a different operating year rather than to a held-out test set from the same pretraining-year distribution. The reconstruction errors are substantially higher on LANSCE-2025, indicating a temporal distribution shift in the sensor statistics, operating conditions, or available channel coverage.

To further assess cross-facility generalization, we evaluate *GMPS-Booster-Fermilab*, a previously unseen accelerator facility excluded entirely from pretraining. Without any facility-specific fine-tuning, SensOFormer achieves zero-shot imputation on GMPS-Booster, with the B variant obtaining the lowest reconstruction error across all four masking ratios. At 15% masking, the MAE decreases from 154.46 for XTi to 148.07 for B. These results extend our evaluation to demonstrate zero-shot transfer to an unseen accelerator facility with a different sensor configuration.

Qualitative reconstructions in Figs. 4 – 13 (Appendix D) further support the quantitative results of Table 2. The predicted values at masked locations closely follow the normalized ground-truth trajectories in representative held-out windows, showing that the model is not only reducing aggregate

Table 2: Zero-shot imputation performance on held-out test sets, reported as raw (un-normalized scale) mean absolute error (MAE) averaged over masked entries for all the samples in the test set. *Lower values indicate better reconstruction*. Bold values denote the best model for each dataset and mask ratio.

Dataset	Mask ratio	XTi	Ti	Mi	S	B
LANSCE 2019–2024	5%	2.4928	1.9969	2.0296	1.8414	**1.7130**
	15%	2.4693	1.9984	1.9526	1.8916	**1.6769**
	25%	2.5411	2.1002	1.9933	2.0337	**1.7580**
	50%	2.6005	2.3238	2.1687	2.1835	**2.0624**
LHC-BL	5%	3.0867	2.8527	2.6608	2.4952	**2.3110**
	15%	3.1752	2.9858	2.8162	2.5671	**2.5179**
	25%	3.1206	3.1030	2.9639	**2.5627**	2.5879
	50%	2.9752	2.9677	3.0199	2.8093	**2.7347**
HVCM-SNS	5%	136.5608	166.7894	134.7735	130.5587	**127.9945**
	15%	160.5342	157.6206	151.5387	**139.9381**	140.3383
	25%	183.9417	189.1077	177.0788	170.1272	**167.9301**
	50%	201.4619	219.4875	198.7487	207.8971	**197.6245**
RFS-LCLS	5%	0.9644	0.8713	0.8095	0.7624	**0.7234**
	15%	0.9726	0.8775	0.8295	0.7489	**0.7220**
	25%	1.0196	0.9111	0.8763	0.8017	**0.7630**
	50%	1.1120	1.0106	1.0497	0.8997	**0.8896**
SRF-CEBAF	5%	0.2306	0.2288	0.2347	0.2177	**0.2099**
	15%	0.2268	0.2278	0.2264	0.2197	**0.2123**
	25%	0.2241	0.2220	0.2228	0.2194	**0.2140**
	50%	0.2247	0.2182	0.2248	0.2208	**0.2137**
LANSCE-2025	5%	16.4059	11.0818	6.7363	6.8370	**6.1174**
	15%	8.4765	7.2481	7.7511	7.3025	**6.6987**
	25%	7.9151	7.3074	8.1885	7.8608	**6.9565**
	50%	8.4266	8.1807	7.9797	**7.4946**	8.6389
GMPS-Booster	5%	154.4794	152.1034	152.9284	151.7627	**148.5698**
	15%	154.4622	150.8483	150.4868	148.5588	**148.0681**
	25%	156.9602	154.9191	154.6353	150.1825	**150.0526**
	50%	158.7238	156.6129	156.7293	155.2808	**154.2139**

MAE but also recovering local temporal structure. Together, the quantitative and qualitative results demonstrate that masked-denoising pretraining learns reusable sensor representations that can be applied directly to missing-data imputation across heterogeneous accelerator datasets.

4.2.2 Anomaly detection task

We formulate supervised anomaly detection as a binary classification problem on the HVCM-SNS dataset. The labeled dataset contains 10,000 waveform samples, split into 8,000 training samples and 2,000 held-out test samples, and the operationally relevant evaluation is performed on 21 induced early-fault test files following the protocol of Radaideh et al. [2023b]. Each input consists of multichannel HVCM waveforms, and the model predicts whether the signal corresponds to normal operation or a faulty condition.

To adapt SensOFormer to this task, we freeze the pretrained set encoder and latent transformer processor and train only a lightweight binary classification head on top of the latent sequence representation. The head is a one-dimensional temporal CNN that operates on the latent sequence $z \in \mathbb{R}^{B\times L\times d}$ with four Conv1D blocks with GELU activations and max pooling after each convolution. The resulting temporal features are summarized using both global max pooling and global mean pooling, concatenated, normalized, and passed through a three-layer MLP before producing a single binary logit. The classification head of XTi contains 0.22M trainable parameters.

We train the detector head for 200 epochs using AdamW with a learning rate of 10^{-5}. On the 8,000/2,000 train/test split, all methods, including SensOFormer, achieve perfect precision, recall, F1, and AUC, making this split insufficient for distinguishing practical early-warning performance. As shown in Table 3, SensOFormer-XTi passes 20 of the 21 induced early-fault tests, matching the strongest prior hierarchical voting ensemble. Scaling to SensOFormer-B improves this result to 21 of

21 tests (100% success rate), outperforming both XTi and the strongest prior method. Since both pretrained backbones are frozen and use the same classification head, this improvement provides evidence that increasing backbone capacity improves downstream diagnostics.

Table 3: All 11 evaluated models, including nine prior models and SensOFormer-XTi and B, achieved perfect precision, recall, F1, and AUC on the 8000/2000 train/test split. The main comparison is the 21 induced early-fault tests, where a test is counted as passed when $\rho < 25\%$ and F1 > 0.8, following Radaideh et al. [2023b].

Model	Passed tests	Success rate (%)
Hierarchical voting ensemble (VE)	**20 / 21**	**95.2**
AdaBoost (AB)	11 / 21	52.4
Extremely randomized trees (ET)	10 / 21	47.6
CNN	9 / 21	42.9
Bagging classifier (BC)	9 / 21	42.9
Random forest (RF)	9 / 21	42.9
Gradient boosting (GB)	9 / 21	42.9
Linear SVM (LSVM)	5 / 21	23.8
RBF-SVM	5 / 21	23.8
SensOFormer-XTi (Ours)	**20 / 21**	**95.2**
SensOFormer-B (Ours)	**21 / 21**	**100**

Table 4: Test-set accuracy for CEBAF C100 cavity identification and fault identification. We compare SensOFormer-XTi with prior deep-learning baselines from Shabalina et al. [2020]. SensOFormer-XTi matches the strongest baselines on cavity identification and improves fault identification accuracy from 74.10% to 84.01%.

Task	LSTM	CNN, direct mapping	CNN, scalogram	**SensOFormer-XTi (Ours)**
Cavity identification	84.60	84.60	45.00	**84.80**
Fault identification	74.10	74.10	58.90	**84.01**

4.2.3 Fault and cavity identification task

We evaluate SensOFormer on supervised SRF fault diagnosis using the SRF-CEBAF dataset. The CEBAF C100 cryomodule data support two related multiclass classification tasks: *cavity identification*, where the goal is to identify the first faulting cavity, and *fault identification*, where the goal is to classify the fault type. These tasks are challenging because SRF cavities in a cryomodule are strongly coupled, so a primary fault can quickly induce secondary responses in neighboring cavities.

To adapt SensOFormer to these tasks, we reuse the pretrained encoder–processor backbone and replace the reconstruction decoder with a lightweight temporal CNN classification head. For each event, the backbone produces a latent sequence $z \in \mathbb{R}^{B\times 192\times d}$, where 192 is the event-window length and d is the latent feature dimension. We train separate heads for cavity identification and fault identification, with output dimensions of 9 and 8 classes, respectively. The head first applies layer normalization to the latent sequence, then processes it with two one-dimensional convolutional layers along the temporal dimension, followed by three fully connected layers. This task-specific head contains approximately 0.5M trainable parameters. We train for 200 epochs using AdamW with a learning rate of 10^{-4} and batch size of 32.

Table 4 compares SensOFormer-XTi with prior deep-learning baselines on the CEBAF test set. For cavity identification, SensOFormer-XTi achieves 84.80% accuracy, comparable to the best prior LSTM and CNN direct-mapping baselines. For fault identification, SensOFormer-XTi achieves 84.01% accuracy, substantially improving over the prior 74.10% baselines. These results indicate that the pretrained SensOFormer representation is useful not only for reconstruction, but also for supervised diagnostic tasks that require distinguishing short RF waveform fault signatures.

We further evaluate fault identification in a data-scarce setting, motivated by the fact that accelerator fault data are costly to collect and require expert annotation. To assess label efficiency, we vary the fraction of labeled training examples while keeping the test set fixed. As shown in Figure 2, SensOFormer-XTi improves monotonically as additional labeled data are introduced and surpasses the prior deep-learning benchmark using only 25% of the training set.

Scaling the model from XTi (2.38M parameters) to B (80.54M parameters) consistently improves accuracy across all training fractions, reaching 85.69% compared with 84.01% at full training data. Both models use frozen pretrained backbones with the same classification-head architecture, labeled-data splits, and optimization settings. These results suggest that self-supervised pretraining provides useful representations for label-efficient SRF fault diagnosis, with additional improvements from increased model capacity. Further details of the published diagnostic baseline architectures, evaluation protocols, and parameter-count estimates are provided in Appendix B.

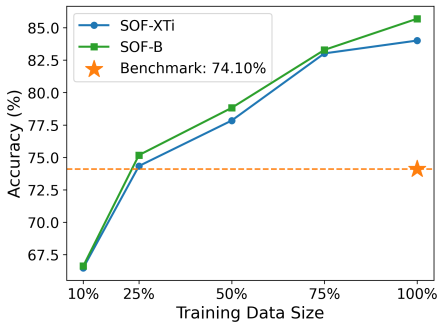


Figure 2: Fault-identification accuracy.

Limitations of current research. Our evaluation covers multiple accelerator facilities and subsystems, but remains limited to the available public and archived datasets. SensOFormer incurs substantial one-time pretraining costs, while larger backbones increase inference costs. Task-specific models may remain preferable for fixed sensor configurations and narrowly defined tasks. For substantial distribution shifts, parameter-efficient fine-tuning may provide a practical adaptation strategy. The present study does not investigate continual learning, which remains an important direction for adapting to changing accelerator operating conditions. In addition, SensOFormer is evaluated offline. Real-time deployment and integration with accelerator control systems remain future work.

5 Conclusion

We introduced SensOFormer, a self-supervised foundation model for particle-accelerator operational data. The design is well suited to accelerator telemetry, where sensors can vary across facilities, subsystems, acquisition windows, and operating modes. By pretraining with masked denoising reconstruction, SensOFormer learns temporal and cross-channel structure from noisy, incomplete, and diverse accelerator measurements. Across datasets from LANSCE, LHC, SNS, LCLS, CEBAF, and Booster, SensOFormer demonstrates strong transfer across missing-data reconstruction, anomaly detection, cavity identification, and fault identification. The scaling study further shows that reconstruction performance improves with both model size and pretraining data, suggesting that accelerator foundation models can benefit from systematic scaling. These results support the feasibility of a general-purpose sensor foundation model for large-scale scientific facilities. Instead of training separate models for each subsystem or diagnostic task, SensOFormer provides a reusable backbone that can be adapted to multiple accelerator problems with limited additional supervision.

Data and code availability

The datasets used in this work are cited in the paper. Publicly available datasets are accessed through their original releases. Any datasets subject to facility-specific access restrictions are not redistributed. Code along with explainer video is available on the project page: `https://mahindrautela.github.io/sensoformer`.

Acknowledgements

Research presented in this article was supported by the Programmatic AI for LANSCE effort (internal charge code: CFLC) through the AOT-IC group at Los Alamos National Laboratory (LANL). This work used computational resources provided by the Darwin testbed at LANL, which is funded by the Computational Systems and Software Environments subprogram of LANL's Advanced Simulation and Computing (ASC) program (NNSA/DOE). Los Alamos National Laboratory is supported by the U.S. Department of Energy National Nuclear Security Administration under Contract No. 89233218CNA000001.

References

A. F. Ansari, L. Stella, C. Turkmen, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.

C. Bodnar, W. P. Bruinsma, A. Lucic, M. Stanley, A. Allen, J. Brandstetter, P. Garvan, M. Riechert, J. A. Weyn, H. Dong, et al. A foundation model for the earth system. *Nature*, 641(8065):1180–1187, 2025.

R. Bommasani, D. A. Hudson, E. Adeli, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

G. Brixi, M. G. Durrant, J. Ku, M. Naghipourfar, M. Poli, G. Sun, G. Brockman, D. Chang, A. Fanton, G. A. Gonzalez, et al. Genome modelling and design across all domains of life with evo 2. *Nature*, pages 1–13, 2026.

M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.

A. W. Chao, M. Tigner, H. Weise, and F. Zimmermann. *Handbook of accelerator physics and engineering*. World scientific, 2023.

A. Das, W. Kong, R. Sen, and Y. Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, pages 10148–10167, 2024.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

A. Edelen and X. Huang. Machine learning for design and control of particle accelerators: A look backward and forward. *Annual Review of Nuclear and Particle Science*, 74(1):557–581, 2024.

A. Edelen, N. Neveu, M. Frey, Y. Huber, C. Mayes, and A. Adelmann. Machine learning for orders of magnitude speedup in multiobjective optimization of particle accelerator systems. *Physical Review Accelerators and Beams*, 23(4):044601, 2020.

C. Garcia-Cardona and A. Scheinker. Machine learning surrogate for charged particle beam dynamics with space charge based on a recurrent neural network with aleatoric uncertainty. *Physical Review Accelerators and Beams*, 27(2):024601, 2024.

K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.

J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

M. Horn, M. Moor, C. Bock, B. Rieck, and K. Borgwardt. Set functions for time series. In *International Conference on Machine Learning*, pages 4353–4363. PMLR, 2020.

R. Humble, F. H. O'Shea, W. Colocho, M. Gibbs, H. Chaffee, E. Darve, and D. Ratner. Beam-based rf station fault identification at the slac linac coherent light source. *Physical Review Accelerators and Beams*, 25(12):122804, 2022.

A. Jaegle, S. Borgeaud, J.-B. Alayrac, C. Doersch, C. Ionescu, D. Ding, S. Koppula, D. Zoran, A. Brock, E. Shelhamer, et al. Perceiver io: A general architecture for structured inputs & outputs. *arXiv preprint arXiv:2107.14795*, 2021.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

E. Krymova, G. Obozinski, M. Schenk, L. Coyle, and T. Pieloni. Data-driven modeling of beam loss in the lhc. *Frontiers in Physics*, 10:960963, 2023.

J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pages 3744–3753. PMLR, 2019.

C. Leon and A. Scheinker. Physics-constrained machine learning for electrodynamics without gauge ambiguity based on fourier transformed maxwell's equations. *Scientific Reports*, 14(1):14809, 2024.

Z. Li, Z. Rao, L. Pan, P. Wang, and Z. Xu. Ti-mae: Self-supervised masked time series autoencoders. *arXiv preprint arXiv:2301.08871*, 2023.

J. Liang, W. Colocho, F.-J. Decker, R. Humble, B. Morris, F. H. O'Shea, D. A. Steele, Z. Zhang, E. Darve, and D. Ratner. Coincident learning for beam-based rf station fault identification using phase information at the slac linac coherent light source. *Physical Review Accelerators and Beams*, 28(12):124601, 2025.

A. A. Mishra, A. Edelen, A. Hanuka, and C. Mayes. Uncertainty quantification for deep learning in particle accelerator applications. *Physical Review Accelerators and Beams*, 24(11):114601, 2021.

M. Moor, O. Banerjee, Z. S. H. Abad, H. M. Krumholz, J. Leskovec, E. J. Topol, and P. Rajpurkar. Foundation models for generalist medical artificial intelligence. *Nature*, 616(7956):259–265, 2023.

T. Nguyen, J. Brandstetter, A. Kapoor, J. K. Gupta, and A. Grover. Climax: A foundation model for weather and climate. In *International Conference on Machine Learning*, 2023.

Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730*, 2022.

D. Park, S. Li, Y. Huang, X. Luo, H. Yu, Y. Go, C. Pinkenburg, Y. Lin, S. Yoo, J. Osborn, et al. Fm4npp: A scaling foundation model for nuclear and particle physics. *arXiv preprint arXiv:2508.14087*, 2025.

N. Park and S. Kim. How do vision transformers work? *arXiv preprint arXiv:2202.06709*, 2022.

M. I. Radaideh, C. Pappas, J. Walden, D. Lu, L. Vidyaratne, T. Britton, K. Rajput, M. Schram, and S. Cousineau. Time series anomaly detection in power electronics signals with recurrent and convlstm autoencoders. *Digital Signal Processing*, 130:103704, 2022.

M. I. Radaideh, C. Pappas, M. Wezensky, and S. Cousineau. Operational data for fault prognosis in particle accelerators with machine learning. *Data in Brief*, 51:109658, 2023a.

M. I. Radaideh, C. Pappas, M. Wezensky, P. Ramuhalli, and S. Cousineau. Early fault detection in particle accelerator power electronics using ensemble learning. *International Journal of Prognostics and Health Management*, 14(1), 2023b.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

M. Rautela and A. Scheinker. Advancing accelerator virtual beam diagnostics through latent evolution modeling: An integrated solution to forward, inverse, tuning, and uq problems. In *Proc. NAPAC2025*, North American Particle Accelerator Conference, pages 38–41. JACoW Publishing, Geneva, Switzerland, 08 2025.

M. Rautela, A. Williams, and A. Scheinker. A conditional latent autoregressive recurrent model for generation and forecasting of beam dynamics in particle accelerators. *Scientific Reports*, 14(1): 18157, 2024.

M. S. Rautela, A. Most, S. Mansingh, B. C. Love, A. Scheinker, D. Oyen, N. Debardeleben, E. Lawrence, and A. Biswas. Morph: Pde foundation models with arbitrary data modality. *arXiv preprint arXiv:2509.21670*, 2025.

R. Roussel, A. Edelen, et al. Multiobjective bayesian optimization for online accelerator tuning. *Physical Review Accelerators and Beams*, 24(6):062801, 2021.

S. Saxena, A. Williams, R. Fierro, and A. Scheinker. Improved robustness of deep reinforcement learning for control of time-varying systems by bounded extremum seeking. *arXiv preprint arXiv:2510.02490*, 2025.

A. Scheinker. Adaptive machine learning for robust diagnostics and control of time-varying particle accelerator components and beams. *Information*, 12(4):161, 2021.

A. Scheinker. Physics-constrained superresolution diffusion for six-dimensional phase space diagnostics. *Physical Review Research*, 7(2):023091, 2025.

A. Scheinker and A. Williams. Latent diffusion can map beam loss to two-dimensional phase-space projections. *Physical Review Accelerators and Beams*, 28(9):094602, 2025.

A. Scheinker, F. Cropp, and D. Filippetto. Adaptive autoencoder latent space tuning for more robust machine learning beyond the training set for six-dimensional phase space diagnostics of a time-varying ultrafast electron-diffraction compact accelerator. *Physical Review E*, 107(4):045302, 2023.

A. Shabalina, A. Carpenter, M. Rahman, C. Tennant, and L. Vidyaratne. Machine learning based cavity fault classification and prediction. Technical report, Thomas Jefferson National Accelerator Facility (TJNAF), Newport News, VA ..., 2020.

J. St. John, C. Herwig, D. Kafkes, J. Mitrevski, W. A. Pellico, G. N. Perdue, A. Quintero-Parra, B. A. Schupbach, K. Seiya, N. Tran, et al. Real-time artificial intelligence for accelerator control: A study at the fermilab booster. *Physical Review Accelerators and Beams*, 24(10):104601, 2021.

C. Tennant, A. Carpenter, T. Powers, A. Shabalina Solopova, L. Vidyaratne, and K. Iftekharuddin. Superconducting radio-frequency cavity fault classification using machine learning at jefferson laboratory. *Physical Review Accelerators and Beams*, 23(11):114601, 2020.

A. Wadell, A. Bhutani, V. Azumah, A. R. Ellis-Mohr, C. Kelly, H. Zhao, A. K. Nayak, K. Hegazy, A. Brace, H. Lin, et al. Foundation models for discovery and exploration in chemical space. *arXiv preprint arXiv:2510.18900*, 2025.

J. Wan, J. Qiang, and Y. Hao. Symplectic machine learning model for fast simulation of space-charge effects. *Physical Review Accelerators and Beams*, 28(7):074602, 2025.

P. Wang, W. Zheng, T. Chen, and Z. Wang. Anti-oversmoothing in deep vision transformers via the fourier domain analysis: From theory to practice. *arXiv preprint arXiv:2203.05962*, 2022.

G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo. Unified training of universal time series forecasting transformers. In *Forty-first International Conference on Machine Learning*, 2024.

A Extended Related Works

Relation to set-based and any-variate time-series models. Set-based processing of irregular observations, arbitrary-variate modeling, and multi-source self-supervised pretraining are established ideas in time-series learning. SeFT [Horn et al., 2020] represents irregularly sampled multivariate time series as an unordered set of observation events and aggregates them with set functions, while MOIRAI [Woo et al., 2024] employs any-variate attention to accommodate an arbitrary number of variates and is pretrained on a large multi-domain corpus for zero-shot forecasting. SensOFormer differs from these models across several architectural, training, and task dimensions, as summarized in Table 5. Our contribution therefore does not lie in any one of these components in isolation. Rather, it lies in integrating latent channel-set compression, temporal representation learning, permutation-equivariant reconstruction, structured telemetry corruption, and multi-facility pretraining for accelerator imputation and diagnostic tasks.

Applicability of MOIRAI to accelerator telemetry. We also examined the released Uni2TS implementation of MOIRAI[1] for zero-shot inference and fine-tuning on accelerator datasets. The released formulation is not directly task-equivalent to our benchmark. Its default fine-tuning configuration sets the maximum number of variates to 128, whereas our datasets contain $C \in [1, 1754]$ channels. A comparable adaptation would therefore require changes to the masking pipeline, output heads, losses, and high-dimensional channel handling, as summarized in Table 6. More broadly, accelerator telemetry combines facility-dependent sensor layouts, up to 1,754 channels, heterogeneous sampling, structured missingness and noise, and limited diagnostic labels; SeFT and MOIRAI are therefore not directly applicable without changing their input representation, training objective, and output modules.

Table 5: Comparison of SensOFormer with the set-based irregular time-series model SeFT [Horn et al., 2020] and the any-variate time-series foundation model MOIRAI [Woo et al., 2024].

	SeFT	**MOIRAI**	**SensOFormer (Ours)**
Input token	Observation event (t, z, m)	Time–variate patch	Sensor-channel set at each time step
Channel encoder	Independent MLPs with Deep-Set aggregation	Any-variate full self-attention	Perceiver-style latent set encoder with cross-attention
Temporal modeling	No sequential processor; time is encoded independently for each event	Joint time–variate encoder Transformer	Separate latent temporal Transformer
Decoder / output	Attention pooling followed by a classification MLP	Linear patch projection to probabilistic mixture parameters	Permutation-equivariant set decoder with cross-attention
Self-supervised pretraining	✗	✓	✓
Training objective	Supervised classification	Self-supervised probabilistic forecasting	Self-supervised denoising masked reconstruction
Masking / corruption	✗	Future-horizon patch masking	Point, time, channel, and block masking with noise corruption
Training data	Three clinical datasets	LOTSA: 27 billion observations across nine domains	Five particle-accelerator facilities
Data characteristics	Irregular and asynchronous measurements with approximately 37–40 modalities	Multiple regular sampling frequencies with up to 128 variates	Tensorized heterogeneous data with 1–1754 sensor channels
Scales to $> 10^3$ channels	✗	✗	✓
Downstream tasks	Mortality and sepsis classification	Zero-shot probabilistic forecasting	Imputation and diagnostic classification

[1] `https://github.com/SalesforceAIResearch/uni2ts`

Table 6: Differences between the released MOIRAI (Uni2TS) formulation and the requirements of our accelerator benchmark.

	Released MOIRAI (Uni2TS)	**Accelerator benchmark requirement**
Task	Predicts a future time-series horizon.	Reconstructs missing values occurring anywhere in the sensor sequence.
Masking	Masks complete future patches.	Uses point-wise, time-wise, channel-wise, and block-wise masks.
Mask granularity	Prediction masks operate at the patch-token level.	Missingness is defined for individual sensor values.
Output head	Uses a probabilistic forecasting output head.	Requires anomaly, cavity, and fault-classification heads.
Number of variates	The default fine-tuning configuration supports at most 128 variates.	Accelerator samples contain as many as 1,754 sensor channels.
Attention	Flattens time–variate patches and applies full self-attention.	Requires efficient processing of very large and variable sensor sets.
Input structure	Fine-tuning examples use fixed-length, fixed-shape sliding windows.	Channel counts and sequence structures differ across facilities.
Overall scope	Natively supports probabilistic future-horizon forecasting.	Our benchmark requires arbitrary-gap imputation and diagnostic classification over heterogeneous datasets with up to 1,754 channels.

B Experimental settings

Model variants. We evaluate five SensOFormer variants, summarized in Table 7. The variants differ in the latent attention dimension, number of attention heads, and transformer depth, while keeping the same encoder–processor–decoder architecture. For all variants, the MLP dimension is set to $4d$, where d is the attention dimension. Unless otherwise specified, the set encoder uses 16 latent slots, the set decoder uses 32 memory slots, gated pooling is used in the set encoder, and the transformer uses absolute positional encodings with maximum length 1024. We restrict the largest model to the B-scale variant based on the scaling study in Section C, where the performance gains begin to show diminishing returns relative to the additional training cost.

Table 7: SensOFormer model variants used in this study. The MLP dimension is set to $4d$ for all variants, where d is the attention dimension.

Model	Attention dim. d	Heads	Depth	MLP dim.	Parameters
XTi	192	4	2	768	2.38M
Ti	256	4	4	1024	5.80M
Mi	384	6	4	1536	13.12M
S	512	8	8	2048	35.87M
B	768	12	8	3072	80.54M

Pretraining data and windowing. SensOFormer is pretrained on the heterogeneous accelerator datasets described in Table 1, including LANSCE, LHC-BL, HVCM-SNS, RFS-LCLS, and SRF-CEBAF data. Each original time series is treated as a separate sequence, and the train/validation split is performed at the sequence level before windowing. This prevents overlapping windows from the same original sequence from appearing in both training and validation sets. Unless otherwise specified, 80% of the available sequences are used for pretraining and 10% each are used for validation and test. Each sequence is then segmented into overlapping windows of length $L = 1024$ with stride 256. The number of sensor channels is kept dataset-specific, and the model is trained without padding along the channel dimension. This is possible because the set encoder and set decoder operate on sensor sets rather than on a fixed global channel layout.

Masking. For each training window, we uniformly sample one mask type from a mask bank and train the model to reconstruct the held-out entries from the remaining visible measurements. The mask bank contains four masking patterns: point-wise, time-wise, channel-wise, and block-wise masking. Point-wise masking independently masks entries with probability 0.15. Time-wise masking

samples one time index and masks a fraction 0.75 of the channels at that time index. Channel-wise masking samples one sensor channel and masks a fraction 0.60 of its time steps. Block-wise masking samples a contiguous rectangular region in the time–channel plane with area approximately $0.15LC$, where L is the window length and C is the number of channels; the block aspect ratio is sampled randomly. For the time-wise, channel-wise, and block-wise masks, we avoid masking an entire row, column, or window whenever possible. These patterns emulate common missingness modes in accelerator telemetry, including isolated dropouts, partial time-slice loss, intermittent sensor-channel loss, and structured channel–time gaps. Masked entries are replaced by a sentinel value, while the binary mask is provided as an input feature so that the model can distinguish intentionally hidden entries from valid observed measurements.

Normalization. Because the pretraining corpus combines sensors with different physical units, dynamic ranges, and operating regimes, we normalize each window on the fly rather than using global dataset statistics. Normalization is performed independently for each batch element and sensor channel, using only the visible entries in that window. Let $x_{b,t,c}$ denote the value for batch element b, time index t, and channel c, and let $M_{b,t,c} = 1$ indicate a masked target. For each (b, c) pair, we compute

$$x_{b,c}^{\min} = \min_{t:M_{b,t,c}=0} x_{b,t,c}, \qquad x_{b,c}^{\max} = \max_{t:M_{b,t,c}=0} x_{b,t,c}, \tag{1}$$

and define the normalization range as

$$r_{b,c} = \max\left(x_{b,c}^{\max} - x_{b,c}^{\min}, \epsilon\right), \tag{2}$$

with $\epsilon = 10^{-5}$. The normalized signal is then

$$\tilde{x}_{b,t,c} = \operatorname{clip}\left(\frac{x_{b,t,c} - x_{b,c}^{\min}}{r_{b,c}}, 0, 1\right). \tag{3}$$

Computing the statistics only from unmasked entries prevents information from the held-out targets from leaking into the input normalization. This per-window, per-channel normalization removes dataset-specific scale differences while preserving the relative temporal structure within each sensor. The same masked normalization procedure is used during validation and downstream imputation.

Noise augmentation. After normalization, we apply denoising augmentation during pretraining. For each training batch, we sample a noise level

$$\sigma \in \{0,\ 0.005,\ 0.01,\ 0.015\}, \tag{4}$$

and add zero-mean Gaussian noise only to the visible entries:

$$\tilde{x}_{b,t,c}^{\text{in}} = \tilde{x}_{b,t,c} + (1 - M_{b,t,c})\epsilon_{b,t,c}, \qquad \epsilon_{b,t,c} \sim \mathcal{N}(0, \sigma^2). \tag{5}$$

Masked entries are not perturbed by this noise process; they are represented to the model through the mask indicator and sentinel value. Including $\sigma = 0$ keeps a subset of batches noise-free, while the nonzero noise levels encourage the model to learn robust denoising representations instead of simply copying the visible measurements. Noise augmentation is used only during training and is not applied during validation or zero-shot imputation.

Training objective. The model is trained to reconstruct the clean normalized signal from the corrupted input. Let x be the normalized target, $\hat{x}$ be the model prediction, and M be the binary mask, where $M_{t,c} = 1$ indicates a masked entry. The training loss is

$$\begin{aligned}\mathcal{L} = &\frac{\sum_{t,c} M_{t,c}(\hat{x}_{t,c} - x_{t,c})^2}{\sum_{t,c} M_{t,c}} + \lambda_{\text{unmask}} \frac{\sum_{t,c}(1 - M_{t,c})(\hat{x}_{t,c} - x_{t,c})^2}{\sum_{t,c}(1 - M_{t,c})} \\ &+ \lambda_{\text{smooth}} \frac{1}{(L-1)C} \sum_{t=1}^{L-1} \sum_{c=1}^{C} |\hat{x}_{t+1,c} - \hat{x}_{t,c}|\,.\end{aligned} \tag{6}$$

The first term is the masked reconstruction loss and is the primary pretraining objective. The second term weakly regularizes reconstruction on unmasked entries, and the third term penalizes high-frequency variation in the predicted sequence. We use $\lambda_{\text{unmask}} = 0.05$ and $\lambda_{\text{smooth}} = 0.01$. Validation loss is computed only on masked entries, matching the missing-data imputation objective.

Optimization and compute. Pretraining is performed using distributed data parallelism on 16 NVIDIA H100 GPUs with 80GB memory each. Each GPU processes one micro-batch, and gradient accumulation is used before applying an optimizer update. With 16 GPUs, a per-GPU micro-batch size of 1, and accumulation over 4 local micro-batches per GPU, the effective global batch size is $16 \times 1 \times 4 = 64$. All SensOFormer variants are pretrained for 25 epochs, where one epoch denotes one pass through the pretraining set. This corresponds to approximately 125K optimizer updates over the full pretraining run. We use the same optimization recipe for all model sizes so that performance differences are primarily attributable to model scale. We use AdamW with learning rate of 10^{-5} and the default PyTorch weight decay. The learning rate follows a warmup-cosine schedule: the first 2.5% of optimizer updates are used for linear warmup, followed by cosine decay over the remaining updates. We use automatic mixed precision with bfloat16 to reduce memory usage and improve throughput, while computing the reconstruction loss in full precision for numerical stability. Gradients are clipped to a maximum norm of 1.0 before each optimizer update. All downstream inference and supervised adaptation experiments were performed on a single NVIDIA RTX A6000 GPU with 48 GB of memory.

HVCM-SNS anomaly-detection baselines. The HVCM-SNS benchmark contains 10,000 pulses with 12 waveforms and 3,753 time samples per pulse, divided into 8,000 training and 2,000 held-out test samples. Published baselines include convolutional neural networks, tree-based ensembles, support vector machines, and a hierarchical voting ensemble. The published CNN comprises four Conv1D blocks followed by three fully connected layers and was trained for five epochs using Adam, a learning rate of 5×10^{-4}, and a batch size of 64. The reported tree-based methods use 40 estimators.

The 21 induced early-fault test files are separate experimental runs excluded from training and standard testing. Each run begins with approximately three minutes of normal operation, followed by gradual perturbation of HVCM settings to induce progressive waveform degradation. Following the published evaluation protocol, a test is counted as passed when the normalized detection delay satisfies $\rho < 25\%$ and $F_1 > 0.8$. On this benchmark, both SensOFormer-XTi and SensOFormer-B use frozen pretrained encoder–processor backbones and the same temporal CNN classification-head architecture, with approximately 0.22M downstream trainable parameters. The full pretrained backbone is not optimized during this supervised adaptation.

CEBAF fault-identification baseline parameter estimates. The published CEBAF baselines consist of a three-layer bidirectional LSTM, a direct-mapping CNN, and a scalogram-input CNN. We reproduce the parameter estimates from the architectures described in [Shabalina et al., 2020]. Since the final classification-layer dimensions are not fully specified, the calculated values are lower bounds, not exact total network parameter counts.

Bidirectional LSTM. The three bidirectional recurrent layers use hidden size $h = 64$. The first layer receives $d = 136$ input features, whereas each of the following two layers receives $d = 128$ features. A bidirectional LSTM layer has

$$P_{\mathrm{BiLSTM}}(d, h) = 2\left(4hd + 4h^2 + 8h\right) \tag{7}$$

parameters, including recurrent and input weights and biases. Consequently,

$$\begin{aligned} P_{\mathrm{LSTM}} &= P_{\mathrm{BiLSTM}}(136, 64) + 2P_{\mathrm{BiLSTM}}(128, 64) && (8)\\ &= 302{,}080 \approx 0.302\,\mathrm{M}. && (9) \end{aligned}$$

Direct-mapping CNN. The reported CNN comprises five convolutional layers, each with 64 output channels, and batch normalization. The first layer has one input channel and the other four layers have 64 input channels. Using the reported 5×5 kernels and including convolutional biases and batch-normalization scale and shift parameters gives

$$\begin{aligned} P_{\mathrm{CNN,direct}} &= (5 \times 5 \times 1 + 1)64 \\ &\quad + 4(5 \times 5 \times 64 + 1)64 \\ &\quad + 5(2 \times 64) && (10)\\ &= 412{,}160 \approx 0.412\,\mathrm{M}. && (11) \end{aligned}$$

Scalogram-input CNN. The scalogram CNN uses the same architecture, except that its first convolution receives 32 input channels. Thus,

$$\begin{aligned} P_{\mathrm{CNN,scalogram}} &= (5 \times 5 \times 32 + 1)64 \\ &\quad + 4(5 \times 5 \times 64 + 1)64 \\ &\quad + 5(2 \times 64) \qquad (12) \\ &= 461{,}760 \approx 0.462\,\mathrm{M}. \qquad (13) \end{aligned}$$

The estimates for all three CEBAF baselines exclude any unspecified final classification layers.

Parameter-count summary and interpretation. Table 8 summarizes the estimated baseline parameter counts alongside the pretrained SensOFormer model sizes and trainable downstream heads. The complete pretrained SensOFormer-XTi and SensOFormer-B models contain 2.38M and 80.54M parameters, respectively (Table 7). Their downstream evaluation uses frozen encoder–processor backbones; the pretraining reconstruction decoder is not part of the classification head. CEBAF classification uses an approximately 0.50M-parameter temporal CNN head, whereas HVCM anomaly detection uses an approximately 0.22M-parameter head. The XTi and B comparisons use the same classification-head designs and training protocols. The exact number of head parameters can depend on the backbone's latent width, so the stated head counts are approximate.

The main controlled comparison is between SensOFormer-XTi and SensOFormer-B: the main change is pretrained backbone scale, while the backbone remains frozen and the classification-head architecture, label fractions, and optimization protocol are held the same. This tests the effect of pretrained representation capacity on downstream discrimination without claiming that the total model parameters match those of the published task-specific baselines. The larger backbone also incurs greater feature-extraction cost, which is not quantified by trainable-head parameter counts alone.

Table 8: Parameter-count comparison for published CEBAF baselines and SensOFormer. Baseline counts are lower bounds inferred from the specified layers; head counts refer only to trainable downstream parameters, while pretrained-model counts refer to the complete pretrained architecture. These categories are not parameter-matched totals.

Model/component	**Parameters**	**Scope**
CEBAF bidirectional LSTM	≥ 0.302M	Specified recurrent layers
CEBAF direct-mapping CNN	≥ 0.412M	Convolutions and batch norm
CEBAF scalogram CNN	≥ 0.462M	Convolutions and batch norm
SensOFormer-XTi	2.38M	Full pretrained model
SensOFormer-B	80.54M	Full pretrained model
SensOFormer HVCM head	~ 0.22M	Trainable head only
SensOFormer CEBAF head	~ 0.50M	Trainable head only

C Scaling studies

Scaling studies are important in the foundation-model setting because they quantify how pretraining loss changes as a function of model size, data size, and training compute. In language modeling, prior work has shown that loss often follows smooth power-law trends with the number of model parameters, training tokens, and compute [Kaplan et al., 2020]. The Chinchilla study further showed that compute-efficient training requires scaling model size and data size together, rather than increasing model capacity alone [Hoffmann et al., 2022]. We perform an analogous scaling analysis for SensOFormer to understand how masked-reconstruction loss depends on the number of model parameters and the amount of pretraining data.

We fit the validation masked-reconstruction loss using a separable scaling law:

$$\mathcal{L}(N, D) = \mathcal{L}_\infty + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \tag{14}$$

where N is the number of trainable model parameters, D is the amount of pretraining data, and $\mathcal{L}(N, D)$ is the validation loss. The constant $\mathcal{L}_\infty$ represents the asymptotic loss approached by the fitted model in the limit of large N and D. The terms A/N^α and B/D^β represent model-limited and data-limited contributions to the loss, respectively. The exponents α and β determine how quickly each contribution decreases as model size or data size increases.

We use this scaling analysis as an empirical diagnostic rather than as an extrapolation guarantee. SensOFormer differs from language models in both data structure and objective: the inputs are heterogeneous accelerator sensor windows rather than text tokens, and the pretraining task is masked denoising reconstruction rather than next-token prediction. Nevertheless, Eq. (14) provides a compact way to summarize whether increasing model capacity and pretraining data improves learned accelerator representations. In our study, we vary N using the five SensOFormer variants in Table 7 and vary D by subsampling the pretraining corpus at 12.5%, 25%, 50%, and 100%. Each scaling run uses single-pass pretraining, and the loss is evaluated on the validation set. The scaling study is performed with an effective batch size of 64 on 32 NVIDIA H100 GPUs.

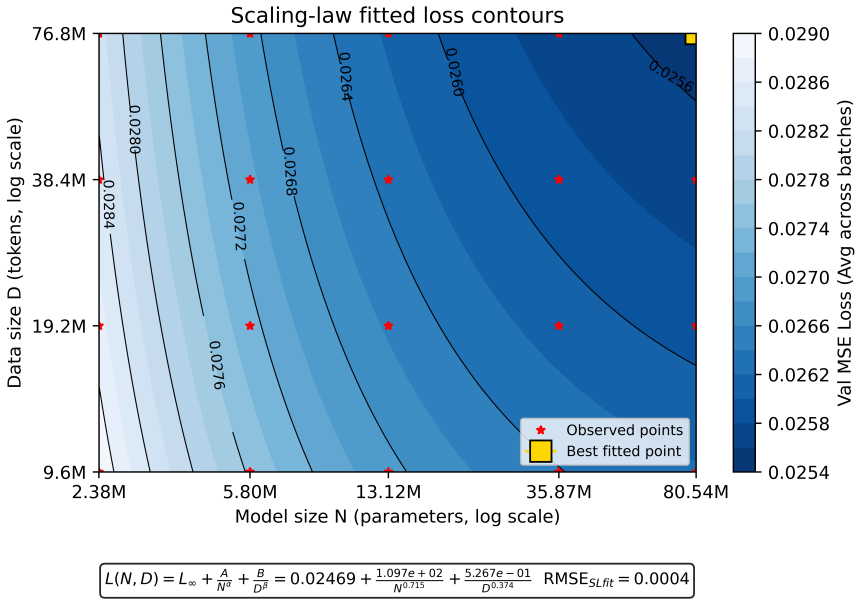


Figure 3: Scaling-law fit for SensOFormer validation masked-reconstruction loss as a function of model size N and pretraining data size D. Red markers denote observed model–data configurations, and the yellow marker denotes the best fitted point within the evaluated grid. The fitted surface shows decreasing loss with both model capacity and data size.

Figure 3 shows the fitted scaling surface obtained by nonlinear least squares. The fitted loss is

$$\mathcal{L}(N, D) = 0.02469 + \frac{1.097 \times 10^2}{N^{0.715}} + \frac{5.267 \times 10^{-1}}{D^{0.374}}, \tag{15}$$

with an RMSE of 4×10^{-4} over the observed model–data configurations. Here, N and D are measured in raw counts rather than in millions. The low RMSE indicates that the separable power law fits the measured validation losses well within the evaluated grid. We do not interpret this as a guarantee that the same law will hold outside the measured range.

The fitted surface shows that validation loss decreases as both model size and data size increase. The best fitted point lies at the largest evaluated model and data scale, indicating that SensOFormer continues to benefit from scale in this regime. The contour geometry also shows that increasing model size produces a strong loss reduction across the evaluated range, especially when moving from the smallest models to the larger S and B variants. Increasing data size also improves loss, but the fitted exponent for data is smaller than that for model size, with $\beta = 0.374$ and $\alpha = 0.715$. This means that, within the fitted law, the data-limited term decays more slowly with additional data than the model-limited term decays with additional parameters. Thus, although the current evaluated regime still benefits strongly from increasing model capacity, future compute-efficient scaling should increase both model size and pretraining data rather than scaling parameters alone.

We can use the fitted law to derive the compute-optimal allocation of model size and data size [Hoffmann et al., 2022]. For a fixed training recipe, let the training compute budget be approximately proportional to the product of model size and data size,

$$\mathcal{C} = \kappa N D, \tag{16}$$

where κ absorbs architecture- and implementation-dependent constants. Let $K = \mathcal{C}/\kappa$, so the compute constraint is $ND = K$. Ignoring the constant $\mathcal{L}_\infty$, the compute-optimal allocation solves

$$\min_{N,D} \left(\frac{A}{N^\alpha} + \frac{B}{D^\beta} \right) \quad \text{s.t.} \quad ND = K. \tag{17}$$

Substituting $D = K/N$ and differentiating with respect to N gives the optimality condition

$$\alpha A (N^\star)^{-\alpha} = \beta B (D^\star)^{-\beta}. \tag{18}$$

This condition states that, at the compute-optimal point, the marginal benefit of increasing model size is balanced against the marginal benefit of increasing data size under the fixed-compute constraint. Solving for the optimal model and data scales gives

$$N^\star(K) = \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha+\beta}} K^{\frac{\beta}{\alpha+\beta}}, \qquad D^\star(K) = \left(\frac{\beta B}{\alpha A} \right)^{\frac{1}{\alpha+\beta}} K^{\frac{\alpha}{\alpha+\beta}}. \tag{19}$$

Using the fitted exponents from Eq. (15), we obtain

$$N^\star \propto \mathcal{C}^{0.343}, \qquad D^\star \propto \mathcal{C}^{0.657}. \tag{20}$$

Under the fitted scaling law, additional compute should therefore be allocated to both larger models and more pretraining data. However, the compute-optimal trajectory increases data size faster than parameter count as the compute budget grows. This is consistent with the broader lesson from Chinchilla-style scaling: larger models are most useful when accompanied by sufficient pretraining data. For SensOFormer, the scaling study suggests that future accelerator foundation models should expand the pretraining corpus alongside model capacity to remain compute-efficient.

D Extended Results

D.1 LANSCE 2019-2024 data

D.1.1 Point-wise missing data imputation

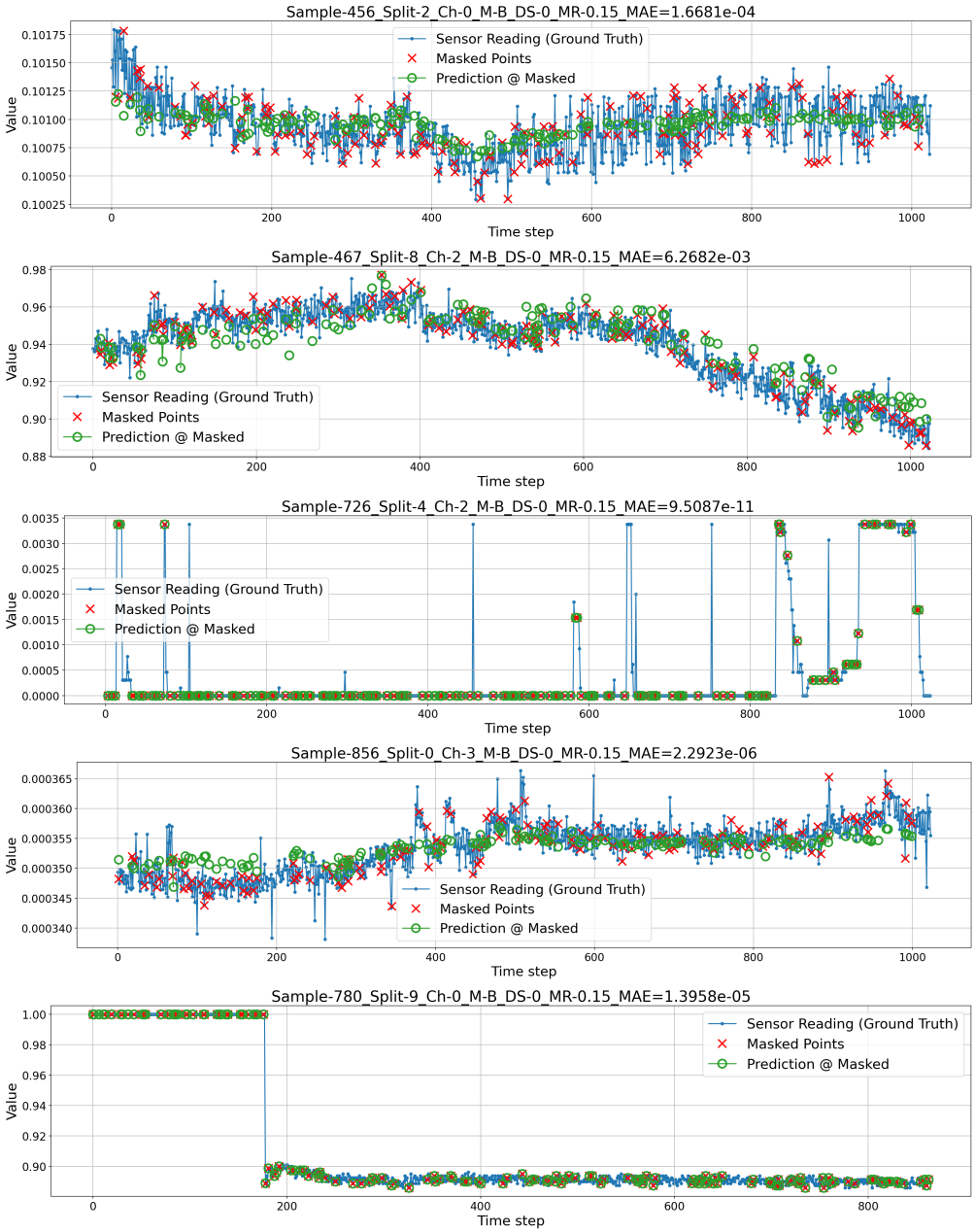


Figure 4: Representative zero-shot *point-wise imputation* examples from pretrained *SensOFormer-B* on held-out *LANSCE 2019–2024* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.1.2 Block-wise missing data imputation

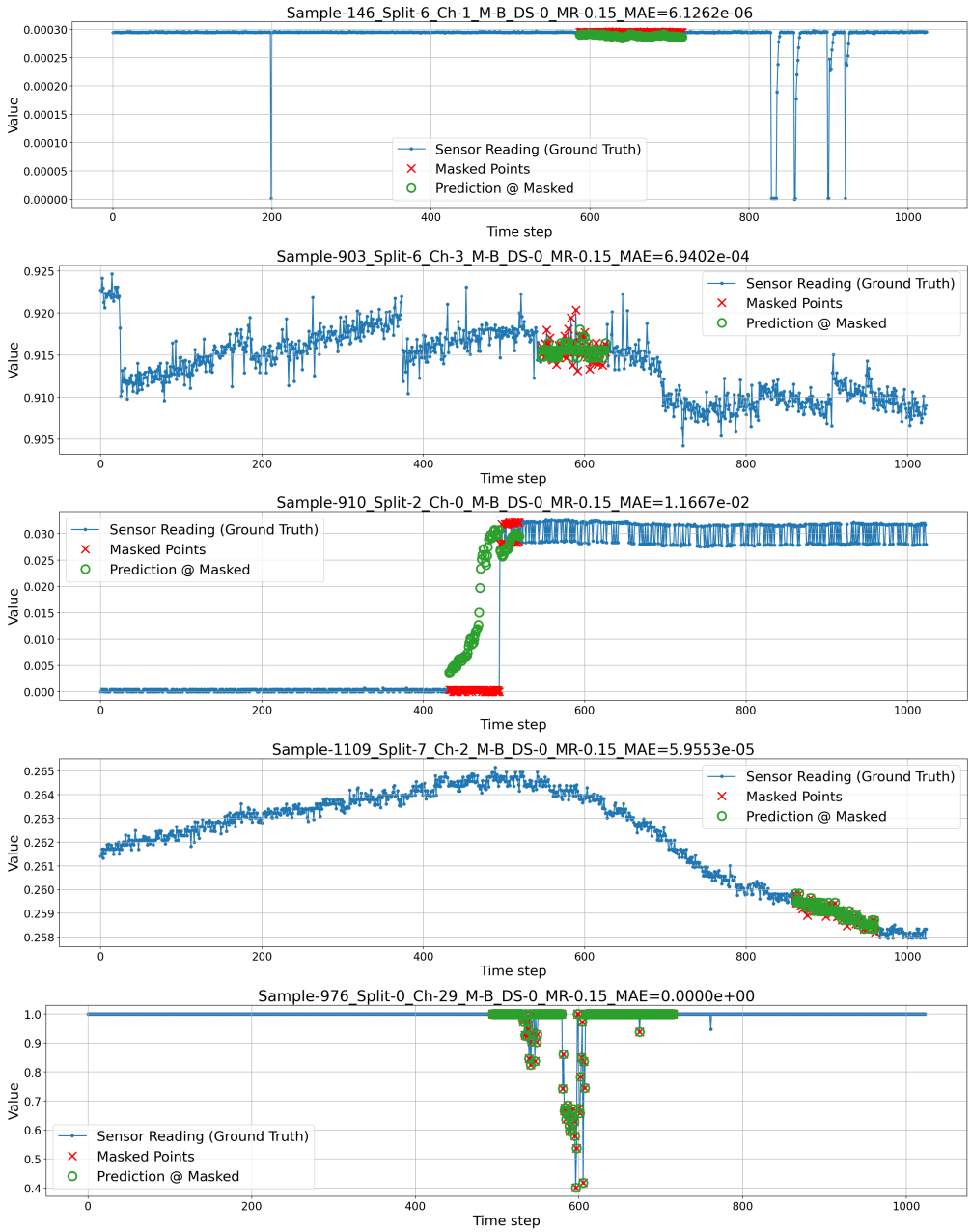


Figure 5: Representative zero-shot *block-wise imputation* examples from pretrained *SensOFormer-B* on held-out *LANSCE 2019–2024* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.2 LANSCE 2025 data

D.2.1 Point-wise missing data imputation

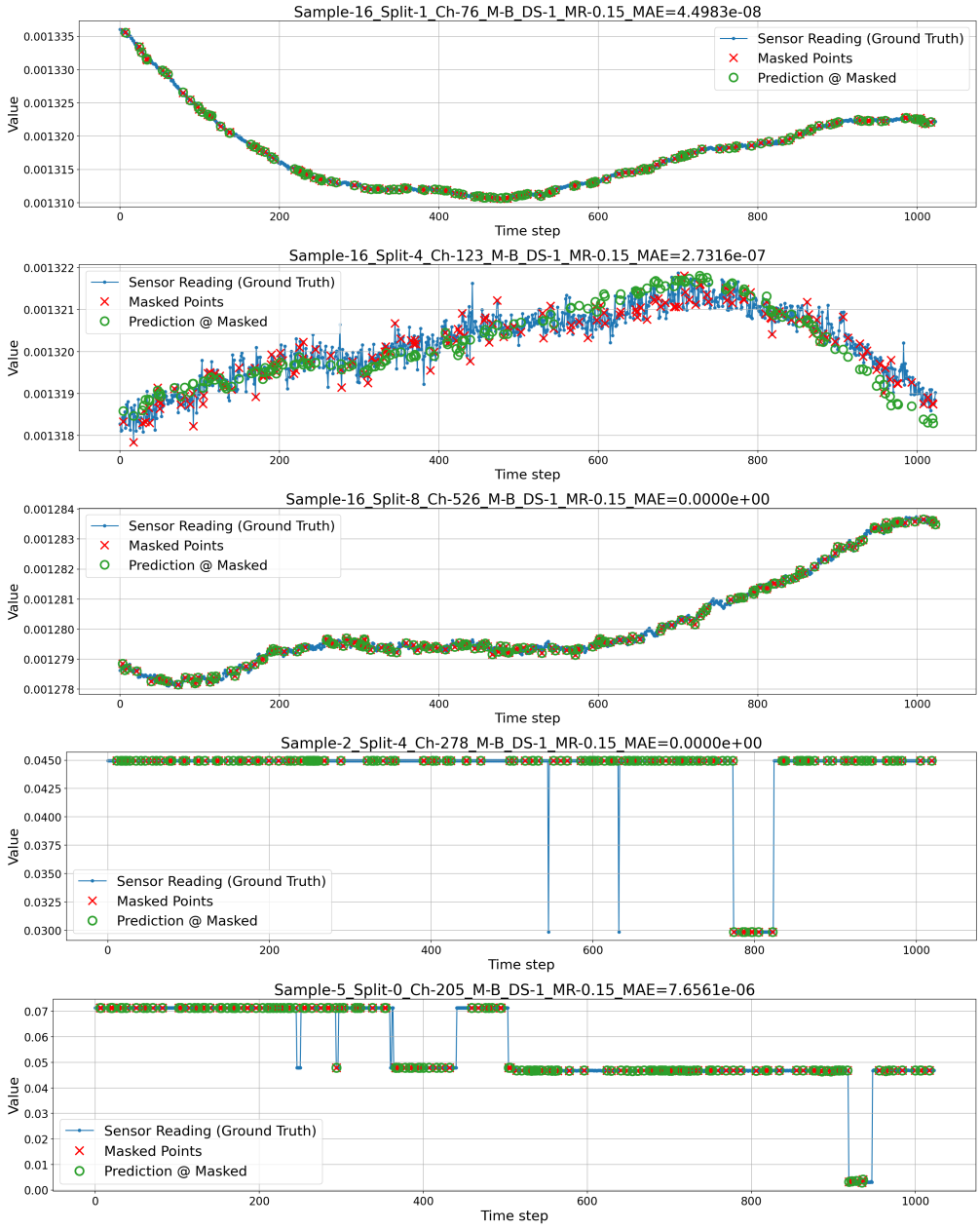


Figure 6: Representative zero-shot *point-wise imputation* examples from pretrained *SensOFormer-B* on held-out *LANSCE 2025* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.2.2 Block-wise missing data imputation

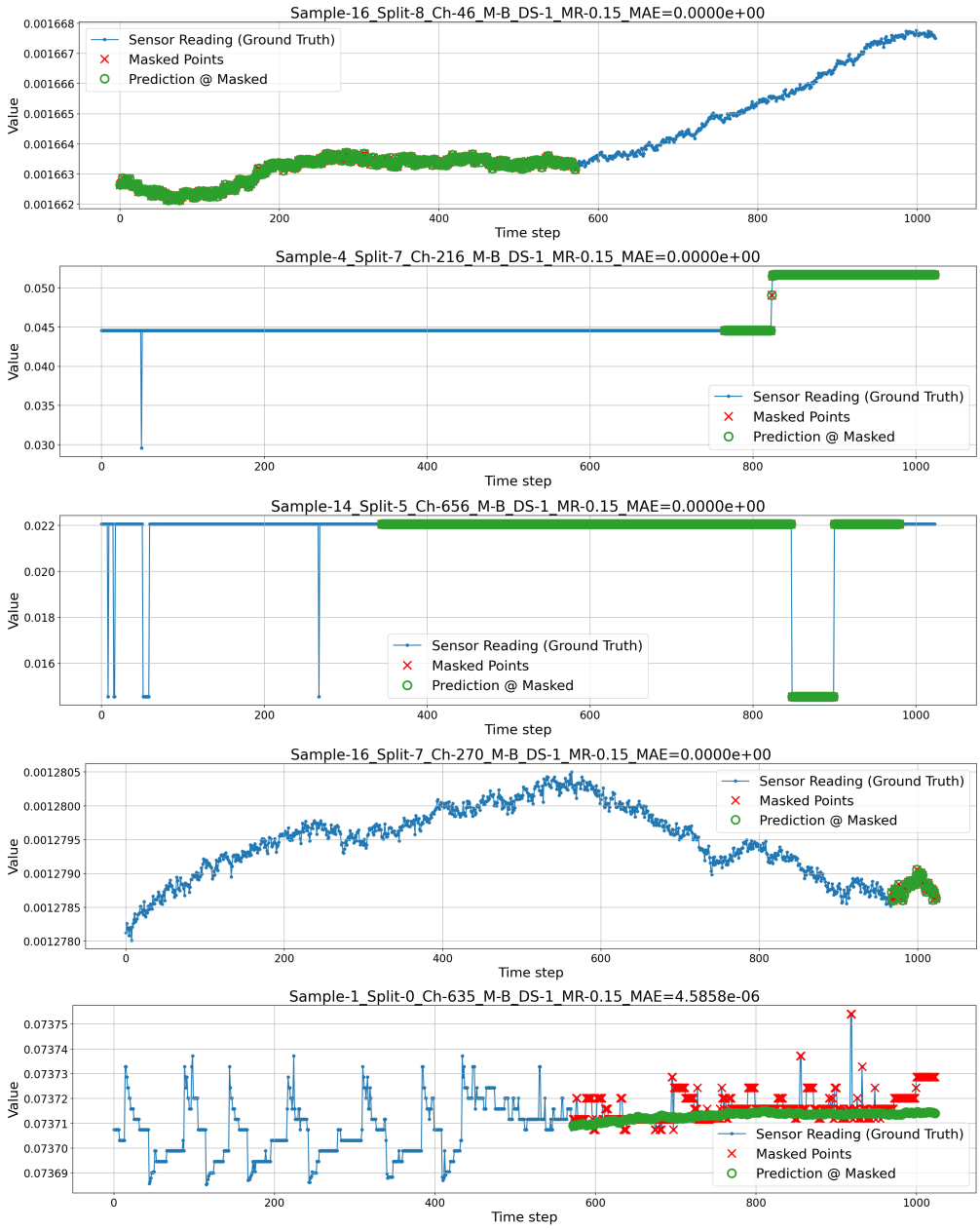


Figure 7: Representative zero-shot *block-wise imputation* examples from pretrained *SensOFormer-B* on held-out *LANSCE 2025* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.3 LHC-BL data

D.3.1 Point-wise missing data imputation

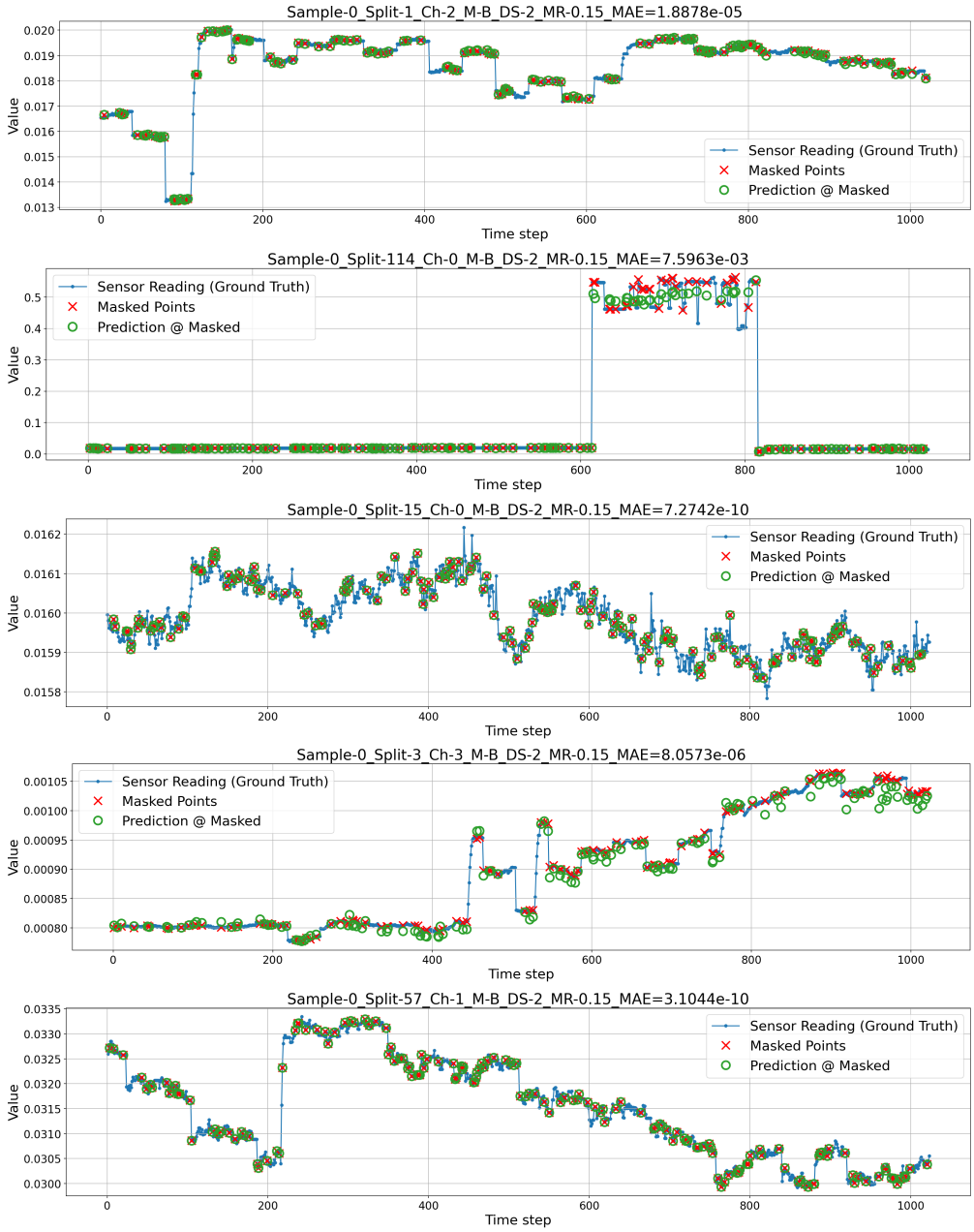


Figure 8: Representative zero-shot *point-wise imputation* examples from pretrained *SensOFormer-B* on held-out *LHC-BL* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.4 HVCM-SNS data

D.4.1 Point-wise missing data imputation

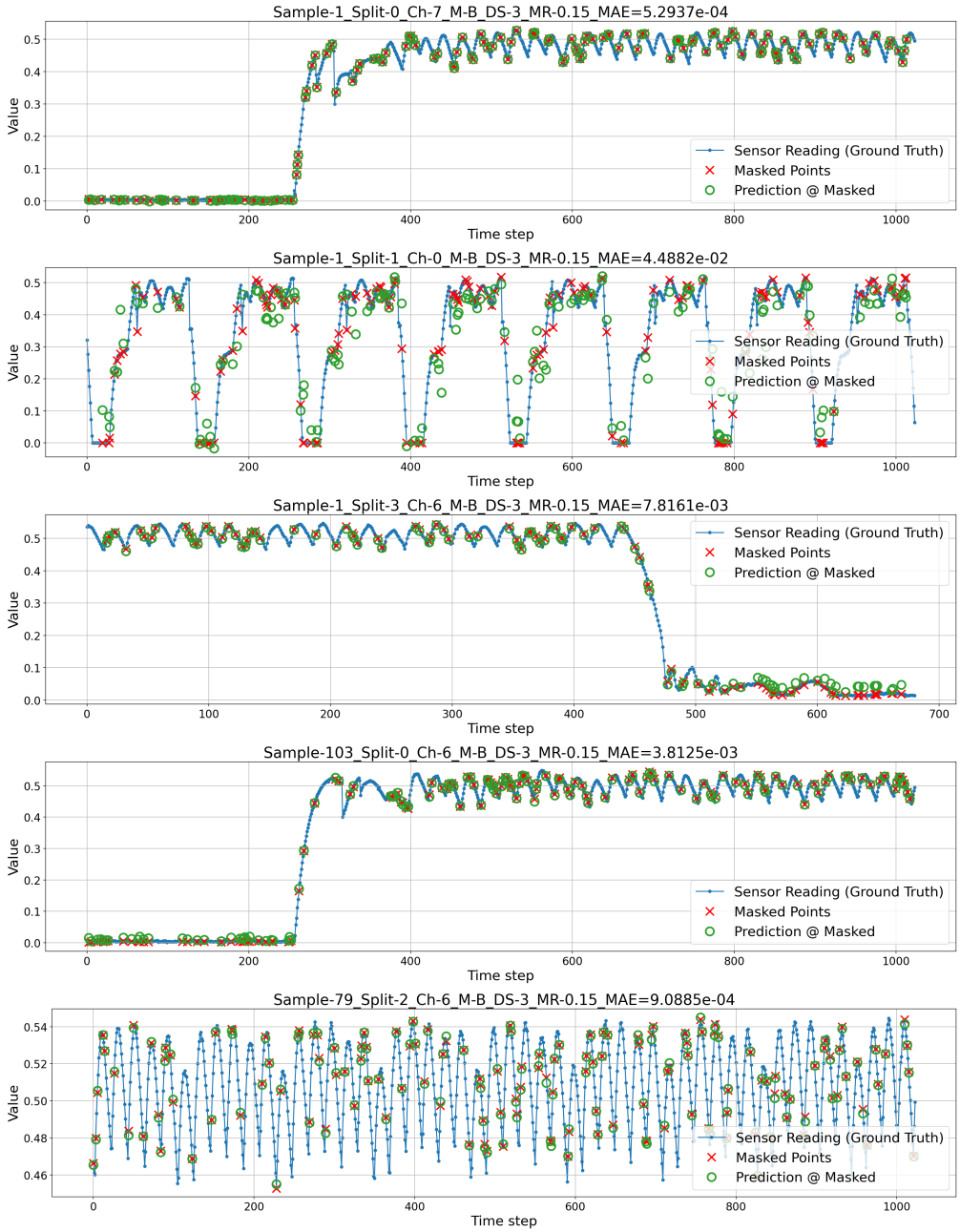


Figure 9: Representative zero-shot *point-wise imputation* examples from pretrained *SensOFormer-B* on held-out *HVCM-SNS* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.4.2 Block-wise missing data imputation

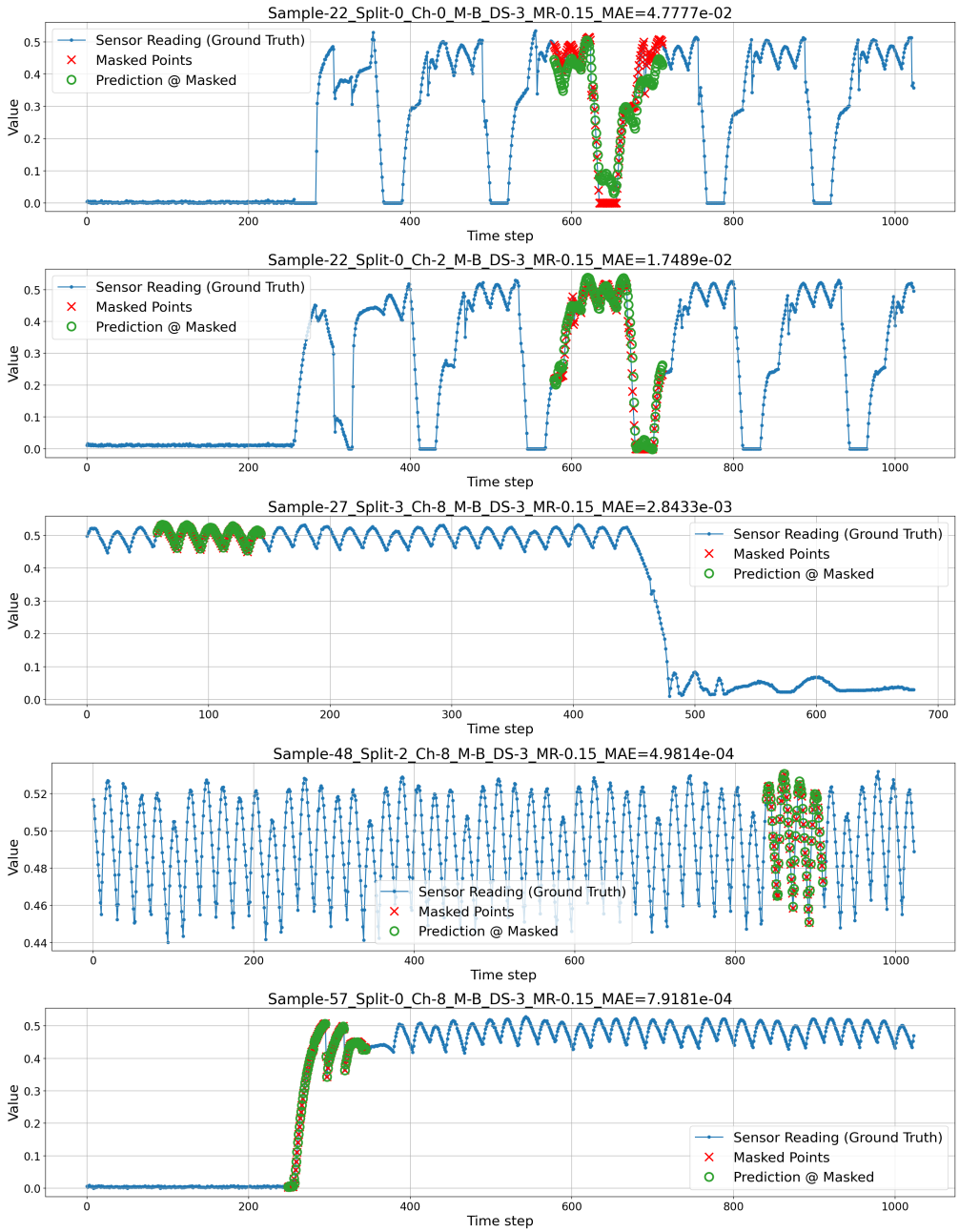


Figure 10: Representative zero-shot *block-wise imputation* examples from pretrained *SensOFormer-B* on held-out *HVCM-SNS* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.5 RFS-LCLS data

D.5.1 Point-wise missing data imputation

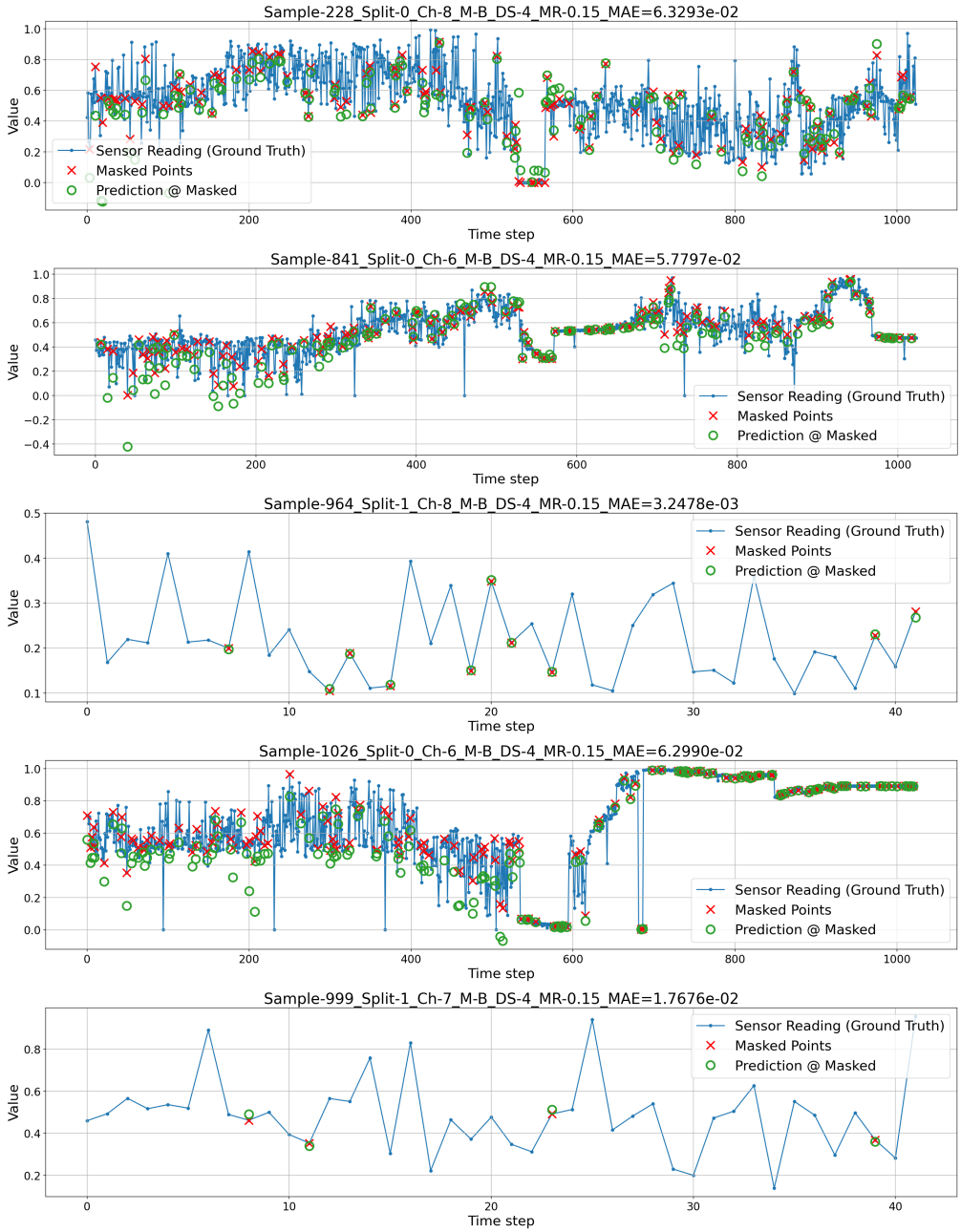


Figure 11: Representative zero-shot *point-wise imputation* examples from pretrained *SensOFormer-B* on held-out *RFS-LCLS* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.5.2 Block-wise missing data imputation



Figure 12: Representative zero-shot *block-wise imputation* examples from pretrained *SensOFormer-B* on held-out *RFS-LCLS* test windows with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

D.6 GMPS-Booster-Fermilab

D.6.1 Block-wise and Point-wise missing data imputation

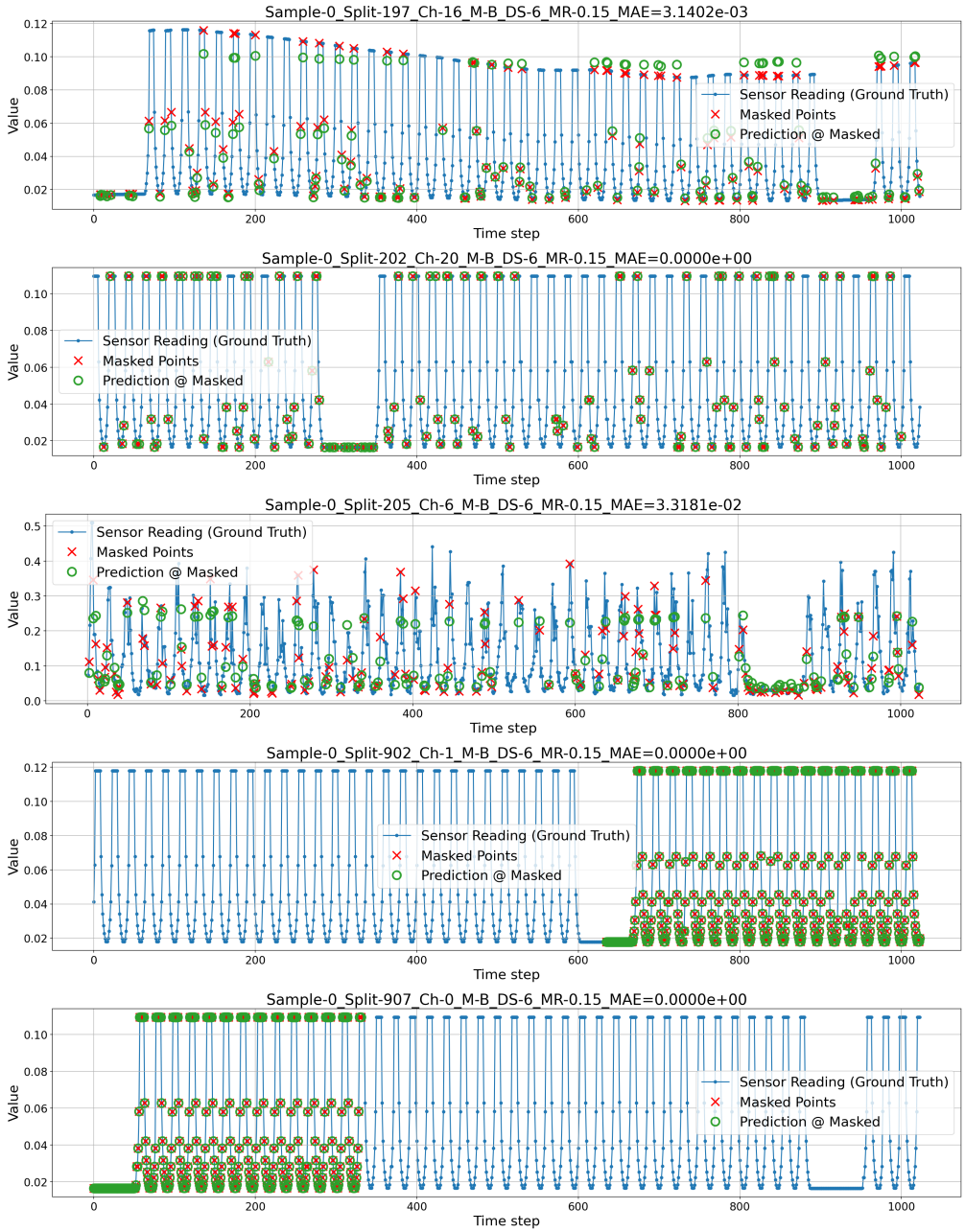


Figure 13: Representative zero-shot *block-wise and point-wise imputation* examples from pretrained *SensOFormer-B* on unseen *GMPS-Booster-Fermilab* facility with a *15% mask ratio*. Blue curves show the normalized ground-truth sensor values, red markers indicate masked entries, and green circles show the model predictions at the masked locations. Each panel corresponds to a different test window and channel. The subplot title reports the sample index, split, channel, model size (M), dataset (DS), mask ratio (MR), and mean absolute error (MAE) computed on the masked entries.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the paper's scope: a self-supervised foundation model for accelerator telemetry, a Perceiver-style set encoder–decoder architecture, pretraining across multiple accelerator datasets, and transfer to imputation and diagnostic tasks. The empirical claims are evaluated in Section 4 and Appendix C and D.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We discuss limitations in the Limitations paragraph and note that scaling-law conclusions are empirical and local in Appendix C.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present formal theoretical results or theorems. The scaling-law analysis in Appendix C is used as an empirical diagnostic rather than a theoretical guarantee.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper describes the model architecture, pretraining objective, datasets, train/validation/test splits, masking strategy, normalization, loss function, optimization settings, model variants, and compute setup in Appendix B. These details are intended to support reproduction of the main experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: We do not provide fully open access to all data and code at submission time because some accelerator datasets may be subject to original-source or facility-specific access restrictions. We provide dataset citations, preprocessing details, architecture details, and training/evaluation settings to support reproducibility.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies the datasets, tensor shapes, downstream tasks, model variants, pretraining windowing, masking, normalization, noise augmentation, loss function, optimizer, learning-rate schedule, batch size, training duration, and downstream adaptation settings in Section 3 and Appendix B.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

Justification: We report held-out test metrics averaged over samples and masked entries, but we do not report error bars, confidence intervals, or repeated full-pretraining runs. Repeating the full multi-facility pretraining study over multiple random seeds was computationally expensive, so results are reported for the trained model variants under the stated experimental protocol.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix B reports the compute resources used for pretraining and downstream adaptation, including GPU types, memory, number of GPUs, batch size, gradient accumulation, number of epochs, optimizer updates, mixed precision, and downstream GPU.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses accelerator operational telemetry and published/archived scientific datasets, does not involve human subjects, and does not intentionally create high-risk generative or surveillance systems. We have reviewed the NeurIPS Code of Ethics and do not identify deviations from it.

   Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The work may positively impact accelerator reliability, data quality, and scientific facility diagnostics. Potential negative impacts include overreliance on offline diagnostic models, false alarms or missed faults in operational settings, and inappropriate deployment without real-time validation or human oversight; these risks are partially mitigated by our offline evaluation scope and by identifying real-time deployment as future work.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release a high-risk generative model, scraped human dataset, or model with an obvious path to misuse comparable to pretrained language or image-generation systems. The work focuses on accelerator telemetry modeling and offline scientific diagnostics.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Existing datasets and baselines are cited in Section 4 and the related work.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper introduces a model architecture and empirical study but does not release a new dataset, model checkpoint, or software asset as part of the submission.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing, user studies, or research with human subjects.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work does not involve human subjects, crowdsourcing, or participant data, so IRB approval or equivalent review is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not an important, original, or non-standard component of the core method, experiments, or scientific claims. Any use of writing, editing, grammar, or formatting assistance does not affect the core methodology or experimental results.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.